webMobaLedLib – Installation und Betrieb

Windows · macOS · Linux-Desktop · Raspberry Pi

Datei-Version: 2.86 (2026-10-27), gilt für webMobaLedLib 2.28

Inhalt

Schnellstart für Eilige

1. Welches System, welches Skript?

2. Gemeinsame Grundlagen

2.1 Dateien und Ordnerstruktur
2.2 Die ARDUINO IDE: Version 1.8.19, nicht 2.x
2.3 Bibliotheken, Boards und die Tabelle „Libraries“
2.4 Starten, Beenden, Speichern
2.5 Das Programmsymbol
2.6 Wo liegen meine Daten? (Datenordner)
2.7 Protokoll (Log)
2.8 Kleine Bildschirme: kompaktes Layout
2.9 Bedienung der Tabelle: Makroauswahl, Bearbeiten, Kontextmenü
2.10 Der Patterngenerator
2.11 Vollbild und Adressen der Reiter
2.12 Servo Einstellungen
2.13 DM Servo Einstellungen, Sound Test und DCC Keyboard
2.14 Z21 Simulator
2.15 Sprache der Oberfläche
2.16 Entwicklungsumgebung: ARDUINO IDE oder PlatformIO
2.17 Anzahl der Eingänge (Excel-Programm 3.5.0-K-long)

3. Windows

4. macOS

5. Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)

6. Raspberry Pi (Dauerdienst)

6.1 Dateien auf den Pi bringen
6.2 Installation (ein Befehl)
6.3 Das Desktop-Symbol
6.4 ARDUINO IDE und Bibliotheken auf dem Pi
6.5 Alltag auf dem Pi
6.6 Raspberry Pi herunterfahren
6.7 Den Raspberry Pi im Netz finden

7. Aktualisieren und Deinstallieren

7.1 Update über das Programm (Hilfe → Update webMobaLedLib …)

8. Zugriff aus dem Netz und Passwort

9. Fehlersuche

9.1 Alle Systeme
9.2 Windows
9.3 macOS
9.4 Linux-Desktop
9.5 Raspberry Pi
9.6 Wenn nichts mehr geht (Installation beschädigt oder Update unterbrochen)

10. Was nicht oder noch nicht geht

Windows · macOS · Linux-Desktop · Raspberry Pi

Der Programmgenerator läuft als Web-Oberfläche: Das Programm startet auf dem Rechner, an dem der Arduino

per USB hängt, und du bedienst es im Browser (Edge, Chrome, Firefox, Safari). Es ersetzt die bisherige Tk-Version und enthält deren Programmmodule. Diese Anleitung gilt für alle vier Systeme. **Eilige lesen nur den Schnellstart.** Wer es genauer wissen will, liest Kapitel 1 und 2 und danach das Kapitel für sein System. Betatester finden eine Checkliste in `install/BETATEST_Checkliste.md`.

Schnellstart für Eilige

Vorbereitung (für alle Systeme)

1. Die Datei **`webmobaledlib.zip`** aus dem **neuesten Release** herunterladen: https://github.com/haroldlinke/WebMLL/releases (dort beim obersten Eintrag unter „Assets“ auf `webmobaledlib.zip` klicken). Direkter Download der neuesten Version: https://github.com/haroldlinke/WebMLL/releases/latest/download/webmobaledlib.zip
2. Einen **neuen Ordner** **`mobaledlib`** anlegen (zum Beispiel im eigenen Benutzerordner), die ZIP-Datei dort hineinkopieren und **entpacken**: Windows Rechtsklick → *Alle extrahieren* (im vorgeschlagenen Zielpfad ein angehängtes `\webmobaledlib` löschen), Linux und Mac „Hier entpacken“. Danach liegt der Ordner `python` direkt in `mobaledlib`.
3. Die **Komplettinstallation** mit einem Befehl starten (Tabelle unten). Sie richtet alles ein: die Python-Umgebung, die richtige **ARDUINO IDE 1.8.19** (nicht 2.x), die Bibliotheken und ein Symbol zum Starten. Das Fenster zeigt den Fortschritt; es dauert einige Minuten.
4. **Starten:** Symbol **„webMobaLedLib“** (Windows: „webMobaLedLib“) doppelklicken. Nach 20 bis 60 Sekunden öffnet sich der Browser mit dem Programm.

System	Komplettinstallation
Windows 10/11	Ordner `mobaledlib\python\install` öffnen, oben in die Adressleiste des Explorers `cmd` tippen und Enter drücken. Dort eingeben: `install_windows.bat /ide`. Windows fragt nach der Erlaubnis. Wird dabei Python neu installiert, **den PC neu starten** und den Befehl noch einmal eingeben.
Linux (Mint, Debian, Ubuntu)	Terminal: `cd ~/mobaledlib/python/install` und dann `bash install_linux.sh --with-arduino-ide`. Danach einmal ab- und wieder anmelden.
macOS (ab 10.13, wenig getestet, siehe Kapitel 4)	Terminal: `cd ~/mobaledlib/python/install` und dann `bash install_mac.command --with-arduino-ide`.
Raspberry Pi	siehe unten

Die Befehle zum Kopieren (jeweils eine Zeile, im Terminal bzw. in der Eingabeaufforderung im Ordner `install`):

```
# Windows (Eingabeaufforderung im Ordner mobaledlib\python\install)
install_windows.bat /ide

# Linux (Mint, Debian, Ubuntu)
cd ~/mobaledlib/python/install
bash install_linux.sh --with-arduino-ide

# macOS
cd ~/mobaledlib/python/install
bash install_mac.command --with-arduino-ide
```

Tipp zum Eingeben: Die Dateinamen enthalten einen Unterstrich (`_`). Beim Kopieren aus einem PDF kann es passieren, dass der Befehl im Terminal auf **zwei Zeilen** erscheint oder der Unterstrich fehlt, dann meldet das Terminal nur einen Fehler. In dem Fall den Namen nicht kopieren, sondern **vorne eintippen und mit der Tab-Taste ergänzen lassen**: `bash install_m` tippen (Linux: `bash install_l`, Pi: `sudo bash install_p`), dann die **Tab-Taste** drücken. Das Terminal schreibt den Dateinamen zu Ende, danach `--with-arduino-ide` anhängen und Enter drücken.

Raspberry Pi: Besonderheiten und einfachster Weg

Der Pi läuft als **Dienst im Dauerbetrieb**: Das Programm startet beim Hochfahren und ist von jedem Rechner im Netz bedienbar (auch ohne Bildschirm am Pi).

1. Mit dem *Raspberry Pi Imager* das neue Betriebssystem **Raspberry Pi OS „Trixie“, 64-Bit, mit Desktop**

auf die SD-Karte schreiben. Im Imager Benutzername, Passwort, WLAN und „SSH aktivieren“ einstellen. Pi starten.

2. Auf dem Pi im Browser die Datei `webmobaledlib.zip` herunterladen (derselbe Link wie oben), im Dateimanager einen **neuen Ordner** **`mobaledlib`** anlegen, die ZIP-Datei hineinkopieren und **„Hier entpacken“**.

3. Terminal öffnen und die Komplettinstallation starten:

   ```
   cd ~/mobaledlib/python/pi
   sudo bash install_pi.sh --with-arduino-ide
   ```

4. Danach: Desktop-Symbol **„webMobaLedLib“** (öffnet das Programm auf dem Pi). Von einem anderen Rechner im Netz: `http://raspberrypi.local:8080` (statt `raspberrypi` der im Imager gewählte Rechnername), Benutzer `mobaledlib`, Passwort `mobaledlib` (im Programm unter „Programm Einstellungen“ änderbar). Der Pi wird über das Menü **Datei → Raspberry Pi herunterfahren** ausgeschaltet, nicht durch Ziehen des Steckers.

Mehr dazu, Optionen und die Fehlersuche stehen in den folgenden Kapiteln.

1. Welches System, welches Skript?

System	Kapitel	Installationsskript (im Projektordner)	Läuft als	Starten
Windows 10 / 11	3	`install\install_windows.bat`	Programm im Konsolenfenster	Verknüpfung „webMobaLedLib“ auf dem Desktop
macOS 10.13 oder neuer	4	`install/install_mac.command`	App `webMobaLedLib.app`	Programm auf dem Schreibtisch
Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch ...)	5	`install/install_linux.sh`	Programm im Benutzerkonto	Symbol im Anwendungsmenü und auf dem Schreibtisch
Raspberry Pi (Dauerbetrieb, auch ohne Bildschirm)	6	`pi/install_pi.sh`	Systemdienst, startet beim Hochfahren	Desktop-Symbol, oder Browser von einem anderen Rechner

Gemeinsam ist allen: Der Ordner **`install`** (und für Linux und Pi der Ordner **`pi`**) liegt **im Projektordner, neben `webmobaledlib.py`**. Die Skripte bestimmen den Projektordner selbst als den Ordner über ihrem eigenen. Es muss kein Pfad eingetragen werden.

2. Gemeinsame Grundlagen

2.1 Dateien und Ordnerstruktur

Der **gesamte Projektordner** **`python`** (mit `pyMobaLedLib.py`, `webmobaledlib.py` und allen anderen Programmdateien) liegt auf dem Rechner, auf dem das Programm läuft. Er enthält die Unterordner `install` (Skripte für Windows, macOS und Linux) und `pi` (Skripte für den Raspberry Pi; das Skript für die Bibliotheken wird auch unter Linux benutzt). Nicht nötig: `__pycache__`. (Konfiguration, Sicherungen und Protokolle liegen nicht im Projektordner, siehe 2.6.)

Das Programm wird mit **`webmobaledlib.py`** gestartet (das erledigen die Starter-Symbole). Im Projektordner liegen außerdem die Programmteile der Web-Oberfläche in den Ordnern **`headless`** und **`_web`**, die angepassten Originaldateien in `ExcelAPI` und `mlpyproggen` sowie die Skripte in `install` und `pi`.

Beispiele für die Lage des Projektordners: `D:\webMobaLedLib\python` (Windows), `~/mobaledlib/python` (Linux, Pi). Die Skripte legen im Projektordner die Python-Umgebung `venv_web` bzw. `venv` an.

2.2 Die ARDUINO IDE: Version 1.8.19, nicht 2.x

Das Programm benutzt die **ARDUINO IDE 1.8.19** zum Kompilieren und Hochladen. Die neue **IDE 2.x wird nicht unterstützt.** Sie steht auf der Startseite von arduino.cc und wird deshalb oft aus Versehen installiert. Eine vorhandene IDE 2.x stört nicht, es wird zusätzlich die Version 1.8.19 gebraucht. Statt der ARDUINO IDE kann auch **PlatformIO** benutzt werden (Abschnitt 2.16).

Am einfachsten installiert sie das Installationsskript mit der passenden Option:

System	Option	Ziel
Windows	`install_windows.bat /ide`	`C:\Program Files (x86)\Arduino` (Installer von arduino.cc, Rückfrage der Benutzerkontensteuerung)
macOS	`bash install_mac.command --with-arduino-ide`	`/Applications/Arduino.app` (ohne Schreibrecht `~/Applications`)
Linux-Desktop	`bash install_linux.sh --with-arduino-ide`	`~/arduino-1.8.19` (ohne root)
Raspberry Pi	`sudo bash install_pi.sh --with-arduino-ide`	`/opt/arduino-1.8.19` (Version für 64-Bit-ARM)

Die Option prüft zuerst, ob die IDE 1.8 schon vorhanden ist, und lädt dann nichts. Danach werden auch die Bibliotheken installiert (Abschnitt 2.3).

Von Hand: Auf arduino.cc unter *Software* im Abschnitt **„Legacy IDE (1.8.x)“** die Version für das System laden (Windows Installer, macOS, Linux 64 bit bzw. ARM 64 bit).

Wie das Programm die IDE findet: Es braucht die **Startdatei der IDE 1.8.x**, unter Linux und macOS die Datei `arduino`, unter Windows `arduino.exe`. Die IDE 2.x und die Version 1.0.5 aus der Paketverwaltung (`apt install arduino`) funktionieren nicht.

- **Windows:** Das Programm sucht selbst in `C:\Program Files (x86)\Arduino` und `C:\Program Files\Arduino`.
- **macOS:** in `/Applications/Arduino.app` und `~/Applications/Arduino.app`.
- **Linux und Raspberry Pi:** in dieser Reihenfolge: ein gültiger Pfad in „ARDUINO Einstellungen“, die Umgebungsvariable `MLL_ARDUINO_EXE`, `/opt/arduino-1.8*/arduino`, `/opt/arduino*/arduino`, `/usr/local/arduino*/arduino`, `~/arduino-1.8*/arduino`, `~/arduino*/arduino`, **eine Ebene tiefer** in `~/*/arduino-1.8*/arduino` (zum Beispiel `~/Downloads/arduino-1.8.19/arduino`) und zuletzt der Suchpfad (`arduino` im Terminal). Von mehreren Treffern gilt der mit der höchsten Nummer. Die Treffer stehen im Protokoll. Wird nichts gefunden, steht dort „ARDUINO IDE nicht gefunden“ mit allen geprüften Orten. Ein Misserfolg wird nur 10 Sekunden gemerkt: Eine danach installierte IDE wird ohne Neustart gefunden.

Am sichersten: Die IDE 1.8.19 nach `~/arduino-1.8.19` entpacken (das macht `install_linux.sh --with-arduino-ide`). Dann findet das Programm sie ohne Einstellung.

Pfad selbst eintragen („ARDUINO Einstellungen“): Die IDE kann irgendwo liegen, dann trägt man den **vollständigen Pfad der Startdatei** ein, zum Beispiel `/home/name/Programme/arduino-1.8.19/arduino` (nicht den Ordner). Ein Dateiauswahlfenster gibt es dafür **nicht**: Die Oberfläche läuft im Browser, und der Browser sieht nur die Dateien seines eigenen Rechners, der Pfad muss aber auf dem Rechner stehen, auf dem das **Programm** läuft (beim Raspberry Pi also auf dem Pi, nicht auf dem PC). Dafür helfen drei Dinge:

- **Prüfung:** Unter dem Feld steht, ob die Datei vorhanden ist, welche Version sie hat und ob der Eintrag falsch ist. Bleibt das Feld leer, steht dort, was die automatische Suche gefunden hat.
- **Schaltfläche „IDE suchen“:** Sie durchsucht bis zu 12 Sekunden lang die üblichen Orte und den Home-Ordner (bis vier Ebenen tief, ohne versteckte Ordner), außerdem `/opt`, `/usr/local` und `/usr/share`. Gefunden wird, was wie eine IDE 1.x aussieht (neben der Datei `arduino` liegen die Ordner `hardware` und `lib`). Mit einem Klick auf einen Treffer wird der Pfad eingetragen, danach die Einstellungen speichern. Die IDE 2.x wird nicht gefunden (sie wird nicht unterstützt).
- **Von Hand im Terminal:** `which arduino` oder `find ~ -maxdepth 5 -type f -name arduino -perm -u+x` zeigt den Pfad. Er lässt sich markieren und einfügen. Im Dateimanager hilft auf vielen Systemen Rechtsklick, **Eigenschaften** oder **Pfad kopieren**.

Der Haken „Individuellen Pfad … verwenden“ ist nur unter Windows nötig. Auf Linux und macOS gilt der eingetragene Pfad immer, wenn die Datei existiert.

2.3 Bibliotheken, Boards und die Tabelle „Libraries“

Zum Kompilieren braucht das Programm die Bibliothek **MobaLedLib** im Sketchbook-Ordner der IDE (Windows `Dokumente\Arduino\libraries`, macOS `~/Documents/Arduino/libraries`, Linux und Pi `~/Arduino/libraries`) und weitere Bibliotheken in bestimmten Versionen, zum Beispiel FastLED. Das Programm prüft die Bibliotheken und Boards

beim Start („Überprüfe Bibliotheken und Boards“) und installiert fehlende in der geforderten Version.

- **MobaLedLib** installiert das Installationsskript mit den Optionen aus 2.2, nachträglich mit `/libs` (Windows), `--with-arduino-libs` (macOS und Linux) bzw. `bash pi/install_arduino_libs.sh` (Pi). Oder in der IDE: *Sketch → Bibliothek einbinden → Bibliotheken verwalten …*, „MobaLedLib“ suchen, *Installieren*.
- **Warum zuerst FastLED:** Die IDE zieht fehlende Abhängigkeiten von MobaLedLib immer in der **neuesten** Version nach, das Programm verlangt aber bestimmte Versionen (zum Beispiel FastLED 3.9.12). Die Skripte installieren deshalb zuerst FastLED in der geforderten Version und danach MobaLedLib. Die Voreinstellung ist 3.9.12, änderbar mit `MLL_FASTLED_VERSION=x.y.z` (Pi und Linux: `MLL_FASTLED_VERSION=x.y.z bash install_arduino_libs.sh`).
- **Kontrolle in der Tabelle „Libraries“:** Sie zeigt die installierten Bibliotheken mit ihrer Version und die vom Programm geforderten Versionen. So öffnest du sie: Button **„Optionen“** in der Symbolleiste → Reiter **„Update“** → Button **„Status der Bibliotheken“**. Die Tabelle „Libraries“ wird dann angezeigt und erscheint wie jede andere Tabelle als Tabellenreiter am unteren Rand. (Die Einstellung „Programmgenerator versteckte Tabellen anzeigen“ in „Programm Einstellungen“ blendet alle ausgeblendeten Tabellen dauerhaft ein; für die Bibliotheken ist sie nicht nötig.) Über der Tabelle stehen die Knöpfe:
  1. **Check actual versions** liest die installierten Versionen neu ein und setzt in der Spalte „Select“ einen **Haken (✓)** bei jeder fehlenden oder falschen Bibliothek (zum Beispiel FastLED 3.10.x statt 3.9.12).
  2. **Install selected** installiert alle Zeilen mit Haken in der geforderten Version neu. Das Programm fragt vorher nach und zeigt den Verlauf im Fenster „Installation der Boards und Bibliotheken“.
  3. **Delete selected** löscht die Bibliotheken bzw. Boards der Zeilen mit Haken.

  Den Haken einer Zeile setzt oder entfernst du mit einem **Klick in die Zelle der Spalte „Select“**.
- **Boards** (zum Beispiel Raspberry Pi Pico) installiert das Programm über dieselbe Tabelle „Libraries“. Boards sind groß (mehrere hundert MB) und brauchen Zeit.
- **ESP32:** Das Programm verlangt das ESP32-Paket in der Version 1.0.4. Auf Windows (Intel/AMD) und Intel-Macs funktioniert das. Auf dem Raspberry Pi und anderen 64-Bit-ARM-Rechnern (auch Macs mit Apple-Chip, nicht geprüft) fehlen für diese Version die Werkzeuge, siehe Kapitel 10. Das Programm wählt das ESP32-Paket auf Linux mit 64-Bit-ARM deshalb **nicht mehr automatisch** aus (ab Version 1.90).
- **Fehlende Bibliotheken beim Start:** Das Programm prüft beim Start und **fragt im Browser** (sobald er verbunden ist), ob fehlende Bibliotheken und Boards installiert werden sollen. Meldungen, die beim Start noch niemand sehen konnte, erscheinen in einem Dialog „Meldungen beim Start des Programms“. Mit „Nein“ geht es ohne Fehlermeldung weiter. Gelingt etwas nicht, wiederholt das Programm einmal nur, **was noch fehlt** (nicht noch einmal alles). Bringt die Wiederholung nichts Neues, endet das Update mit einer Meldung und dem Hinweis auf das Protokoll, ohne Endlosschleife (Hilfe → Logfile anzeigen). Die Platinen sind groß: Internet und Platz auf dem Datenträger prüfen.
- **ESP32 und der Befehl `python`:** Das ESP32-Paket 1.0.4 ruft `python` auf. Neuere macOS-Versionen (ab 12.3) und Debian-Systeme (auch das Raspberry Pi OS) haben nur `python3`. Das Programm legt deshalb beim Start einen Hilfsbefehl `python` im Datenordner an (`bin/python`, ruft den Python des Programms auf), wenn es keinen gibt (ab Version 1.89). Ohne ihn endet das Prüfen und Hochladen mit `exec: "python": executable file not found in $PATH`. Nach dem Einspielen das Programm neu starten.

2.4 Starten, Beenden, Speichern

- Die Oberfläche öffnet sich im Browser unter `http://localhost:8080`. Der Start dauert 20 bis 60 Sekunden. Ist sie schon gestartet, öffnet ein weiterer Doppelklick auf das Symbol nur den Browser.
- **Beenden:** Knopf **„Beenden“** in der Oberfläche (oben rechts). Unter Windows schließt sich dabei das Konsolenfenster. Auf dem Pi startet der Dienst danach von selbst neu.
- Es sollte immer nur **ein Browser** gleichzeitig mit der Oberfläche arbeiten.
- **Kein Tk nötig:** Ab Version 1.33 benutzt das Programm auf **allen** Systemen einen Ersatz für Tk (`no_tk`). Es braucht weder `tkinter` (`python3-tk`) noch ein Fenster auf dem Bildschirm des Servers. Zum Vergleich bei Fehlern startet `MLL_TK_ROOT=1` mit einem unsichtbaren echten Tk-Fenster (nur wenn `tkinter` und ein Bildschirm vorhanden sind, sonst gilt wieder der Ersatz). Das Protokoll nennt die benutzte Art (`Tk: Ersatz no_tk …`).
- **Speichern:** Das Programm speichert beim Beenden („Speichern und beenden“) und zwischendurch automatisch in die Datei `.autosave.json` im Datenordner (2.6). Dateien aus dem Menü „Datei“ werden im Browser heruntergeladen bzw. hochgeladen. Eine PGF-Datei kannst du auch einfach auf die Tabelle ziehen.
- **Meldungen** stehen in der Datei `webmobaledlib.log` im Datenordner (2.6, Einstellungen siehe 2.7), außerdem `webmobaledlib-console.log` bei macOS und Linux und `journalctl` auf dem Pi. Welche Programmversion läuft, steht unter dem Titel der Oberfläche (zum Beispiel „webMobaLedLib 1.16“) und in den Köpfen der Dateien

(„Datei-Version“).

Welcher Browser? Das Programm ist **nicht** auf einen Browser festgelegt. Es öffnet die Adresse im **Standardbrowser des Systems** und prüft, ob das geklappt hat:

System	Reihenfolge
alle	zuerst die Umgebungsvariable `MLL_BROWSER` (Programm, auch mit Argumenten, zum Beispiel `MLL_BROWSER="brave-browser --new-window"`), dann `BROWSER`
Windows	die Standard-App für Webadressen
macOS	`open`
Linux	`xdg-open` (Standardbrowser), `gio open`, `sensible-browser`, `x-www-browser`, danach bekannte Browser: Google Chrome, Chromium, Brave, Edge, Vivaldi, Opera, Firefox, Epiphany, Falkon, zuletzt das Modul `webbrowser` von Python

Endet ein Weg mit einem Fehler (zum Beispiel `xdg-open`, weil kein Standardbrowser eingetragen ist), geht es mit dem nächsten weiter. Der benutzte Weg steht im Protokoll (`Browser geöffnet (xdg-open): http://localhost:8080`). Findet sich gar kein Browser, nennt das Programm die Adresse auf dem Bildschirm und im Protokoll, dann von Hand eingeben. Unter Linux zeigt `xdg-settings get default-web-browser`, welcher Browser als Standard eingetragen ist, und `xdg-settings set default-web-browser google-chrome.desktop` (oder `brave-browser.desktop`) stellt ihn ein.

2.5 Das Programmsymbol

Das Symbol ist das des originalen ProgrammGenerators (Ring aus bunten LEDs, Tabelle, Stift) mit einer kleinen blauen Weltkugel unten rechts. Dieselbe Datei steht als Symbol im Browserreiter. `install/make_icons.py` erzeugt daraus `mobaledlib.png` (Verknüpfungen, Raspberry Pi), `favicon.png`, `mobaledlib.ico` (Windows) und `mobaledlib.icns` (macOS). Ohne Weltkugel: `python make_icons.py --plain`. Die fertigen Dateien liegen bei; die Installer erzeugen sie nur neu, wenn sie fehlen. `make_icons.py` braucht Pillow (`pip install Pillow`), die Web-Oberfläche selbst nicht. Die Vorlage `mobaledlib_source.png` ist nur 128 × 128 Pixel groß und wirkt vergrößert etwas weich; wer sie höher aufgelöst hat, ersetzt die Datei und startet `make_icons.py` noch einmal.

2.6 Wo liegen meine Daten? (Datenordner)

Alles, was das Programm **selbst erzeugt**, liegt **außerhalb des Projektordners** in einem Datenordner des Benutzers. Dadurch bleiben deine Daten erhalten, wenn bei einem Update der Ordner `python` ersetzt oder überschrieben wird.

System	Datenordner
Windows	`%APPDATA%\webMobaLedLib` (das ist `C:\Users\<Name>\AppData\Roaming\webMobaLedLib`)
macOS	`~/Library/Application Support/webMobaLedLib`
Linux, Raspberry Pi	`~/.local/share/webMobaLedLib` (beim Pi für den Benutzer, unter dem der Dienst läuft)

Bis Version 1.30 hieß der Ordner `pyMobaLedLib`. Das Programm benennt ihn beim ersten Start mit der neuen Version selbst um (Konfiguration, Sicherungen, Passwort und Protokolle bleiben erhalten; ist das Umbenennen nicht möglich, wird er kopiert und der alte Ordner darf später gelöscht werden).

Einen anderen Ort legt die Umgebungsvariable **`MLL_DATA_DIR`** fest (zum Beispiel `D:\MobaLedDaten`). Der Ordner wird bei Bedarf angelegt. Wo er ist, zeigen das Fenster beim Start, das Protokoll, der Menüpunkt **Hilfe → Über …** und der Befehl `python userdata.py` im Projektordner.

Was dort liegt: die Konfigurationsdateien (Einstellungen, Farbtest, Palette), die Sicherung der Tabellen (wird beim Beenden mit „Speichern“ geschrieben und beim Start geladen), die Zwischensicherung `.autosave.json`, die Passwort-Datei `web_password.json` (nur als Hash) und die Protokolle `webmobaledlib.log` und `webmobaledlib-console.log`, das Absturzprotokoll `webmobaledlib-crash.log` sowie der Unterordner `nicegui` (interner Speicher der Oberfläche).

Übernahme alter Dateien: Frühe Versionen der Web-Oberfläche (bis 1.22) haben Konfiguration, Tabellensicherung und Passwort-Datei außerhalb des Datenordners abgelegt. Das Programm kopiert sie beim ersten Start einmalig in den Datenordner (die alten Dateien dürfen danach gelöscht werden). **Dateien der Tk-Version werden nicht übernommen:** Die Web-Version teilt keine Dateien mit ihr. Tabellen aus der Tk-Version

lassen sich über eine gespeicherte PGF-Datei laden (auf die Tabelle ziehen).

Was im Projektordner bleibt: das Programm, die Vorlagen und Definitionen und die Python-Umgebung (`venv_web` bzw. `venv`). Wird bei einem Update der ganze Ordner gelöscht und neu entpackt, ist auch die Python-Umgebung weg: dann das Installationsskript noch einmal ausführen.

Sichern und Umziehen: den Datenordner kopieren. **Prüfen, dass nichts im Projektordner landet:** `python install/check_project_writes.py snapshot` vor dem Start, nach einer Sitzung `python install/check_project_writes.py compare` (zeigt neue, geänderte und gelöschte Dateien im Projektordner, mit `--parent` auch eine Ebene darüber).

2.7 Protokoll (Log)

Das Programm schreibt ein Protokoll mit dem Standard-Logger von Python. Es enthält auch die Meldungen der Programmmodule. Die Datei **`webmobaledlib.log`** liegt im Datenordner (2.6). Ohne Angabe wird auf der Stufe **DEBUG** protokolliert, wie im Original. Die Datei dreht sich bei 2 MB (die älteren heißen `webmobaledlib.log.1` bis `.5`), und auf der Konsole (beim Pi im Journal des Dienstes) erscheinen die Meldungen ab INFO. Jede Sitzung beginnt mit einem Kopf aus Python-Version, System, Parametern, Ordnern, Stufe, Dateiname, den Versionen der wichtigsten Pakete und der Umgebung (Sitzungsart, Display, Tk). Außerdem steht darin, wann sich ein Browser verbindet und trennt (`Browser verbunden` / `Browser getrennt`), wie lange der Aufbau der Seite gedauert hat, und eine Warnung, wenn der Server ungewöhnlich lange blockiert war. Bricht der Python-Prozess ab (zum Beispiel durch einen Fehler in einer Systembibliothek), steht der Grund in `webmobaledlib-crash.log`. Auch die Ausgabe der Fenster **„Kompilieren und Hochladen“** und **„Installation der Boards und Bibliotheken“** steht im Protokoll (Logger `webmobaledlib.arduino`, ab Stufe INFO; Zeilen mit Fehlermarkierung als WARNING, jeder Lauf mit einer Kopfzeile). So lässt sich ein Lauf später nachlesen. In der Oberfläche zeigt **Hilfe → Logfile anzeigen** die letzten Zeilen (am Ende steht das Neueste) mit Pfad, Größe und Stufe der Datei. Mit **Aktualisieren** liest es neu, mit **Ganze Datei herunterladen** bekommst du die komplette Datei, zum Beispiel für eine Fehlermeldung.

Kommandozeile	Umgebungsvariable	Wirkung
`--loglevel DEBUG`, `INFO`, `WARNING`, `ERROR` oder `CRITICAL`	`MLL_LOGLEVEL`	Stufe der Protokolldatei (Standard `DEBUG`)
`--logfile NAME`	`MLL_LOGFILE`	Dateiname. Ohne Pfad liegt die Datei im Datenordner, mit absolutem Pfad dort, `stdout` schreibt keine Datei
–	`MLL_DEBUG_TIMING=1`	schreibt die Zeit für Aufbau und Aktualisierung der Tabelle und der Testtasten ins Protokoll (Zeilen mit `TIMING:`), hilft bei der Suche nach langsamen Stellen
–	`MLL_DEBUG_SHAPES=1`	schreibt zu jeder Aktualisierung der Tabelle die Zahl der Symbole und Testtasten und welche Zeilen sie tragen ins Protokoll (Zeilen mit `SHAPES:`), für die Fehlersuche bei fehlenden Symbolen oder Testtasten
–	`MLL_GRID_UPDATE=full`	Die Tabelle wird bei jeder Änderung komplett neu gesendet (wie bis Version 1.48). Zum Prüfen, ob ein Anzeigefehler an der gezielten Aktualisierung liegt; langsamer

Beispiele: Windows `install\start_windows.bat --loglevel INFO`, Linux und macOS `install/start_linux.sh --loglevel INFO`, Raspberry Pi die Zeile `MLL_LOGLEVEL=INFO` in `/etc/mobaledlib-web.env` und danach `sudo systemctl restart mobaledlib-web`. Die sehr gesprächigen Bibliotheken (Webserver, Websocket) bleiben auch bei DEBUG gedämpft. Wer sie sehen will, setzt `MLL_LOG_LIBS=1`. Bei einer Fehlermeldung bitte die Datei `webmobaledlib.log` schicken (und `webmobaledlib.log.1`, falls vorhanden).

2.8 Kleine Bildschirme: kompaktes Layout

Auf kleinen Bildschirmen (zum Beispiel einem Laptop mit 1366 × 768 Punkten, auf dem der Browser nur rund 600 Punkte Höhe hat) bleibt für die Tabelle wenig Platz, weil Kopfzeile, Menü, Reiter und die großen Symbole der Funktionsknöpfe mehr als die Hälfte belegen. Das **kompakte Layout** schafft Platz:

- Menü (Datei, Tabelle, ARDUINO, Hilfe), Titel, Statusmeldung und der Knopf **Beenden** stehen in **einer** Zeile. Der Titel ist kleiner, der Knopf Beenden ohne Symbol, die Versionsangabe und die Zeit der Zwischensicherung sind ausgeblendet (die Version steht unter Hilfe → Über …).
- Die Reiter sind schmaler.
- Die **Symbole der Funktionsknöpfe werden durch ihren Text ersetzt:** eine schmale Reihe kleiner Textknöpfe statt großer Symbole.
- Leerzeilen, Ränder und Abstände entfallen.
- In den **Makrodialogen** (Makroauswahl, Parameterformulare, Haus-Dialog) sind die **Beschreibungstexte auf eine Zeile eingeklappt**. Ein Klick auf den Text (Pfeil ▸) klappt ihn auf, ein weiterer Klick wieder zu.
- Die Zeile „Zeile x letzte benutzte Zeile von y Zeilen“ steht unten rechts neben den Blattreitern statt in einer eigenen Zeile.

Einschalten: Reiter **Programm Einstellungen**, Karte **Darstellung auf kleinen Bildschirmen**, Feld **Kompaktes Layout**:

Stufe	Wirkung
Aus (Standard)	normales Layout
Ein	immer kompakt
Automatisch	kompakt, wenn das Browserfenster niedriger als 800 Pixel ist, und wieder normal, wenn es höher wird

Die Auswahl wirkt sofort, gesichert wird sie mit **Speichern**. Die Einstellung gilt für das Programm auf diesem Rechner (sie steht in der Konfiguration im Datenordner). Ohne gespeicherten Wert gilt die Umgebungsvariable `MLL_LAYOUT` (`normal`, `compact` oder `auto`). Zusätzlich schaffen **Vollbild (F11)** und der **Browser-Zoom** (Strg und Minus) Platz.

2.9 Bedienung der Tabelle: Makroauswahl, Bearbeiten, Kontextmenü

- **Ein Klick auf das Symbol** (Spalte „Icon“) öffnet die Makroauswahl.
- **Zellen von Hand bearbeiten: F2** oder **Enter** auf der markierten Zelle, ein **Doppelklick** (siehe unten) oder im Kontextmenü (Rechtsklick auf die Zelle) **Zelle bearbeiten**. Das ist zum Beispiel in der Spalte „Beleuchtung, Sound, oder andere Effekte“ oft schneller als der Dialog, um einen Parameter zu ändern.
- **Doppelklick auf Name und Beleuchtung:** Wie in der Excel-Version kann ein Doppelklick auf diese beiden Spalten die **Makroauswahl** öffnen. Das ist **standardmäßig aus** (ein Doppelklick bearbeitet die Zelle). Einschalten: Reiter **Programm Einstellungen**, Karte **Bedienung der Tabelle**, Haken **„Doppelklick auf Name und Beleuchtung öffnet die Makroauswahl“**, dann **Speichern**. Er gilt nur in Datenzeilen der Datenblätter, die Kopfzeilen bleiben unverändert.
- **Kontextmenü** (Rechtsklick auf eine Zelle): Zelle bearbeiten (F2), Kopieren (Strg+C), Ausschneiden (Strg+X), Einfügen (Strg+V), Als Zeilen einfügen, Löschen (Entf), Farbe setzen, Farbe entfernen. Sind mehrere Zeilen markiert und man klickt in eine davon, wirken Kopieren, Ausschneiden und Löschen auf die markierten Zeilen.
- **Spalten hinter Q:** Datenblätter zeigen nur die Spalten bis Q. Der Knopf **„Alle Spalten“** (Werkzeugleiste, hinter „Zeile färben“) blendet auch die Spalten dahinter ein, zum Beispiel die LED-Zahlen der Kanäle 1 bis 9 in Zeile 1. **„Spalten bis Q“** schaltet zurück. Die Einstellung gilt bis zum Beenden des Programms.
- **Menü Tabelle → Tabelle neu berechnen:** rechnet **alle** Tabellenblätter neu (LED-Zahlen je Kanal und Start-LED-Nummern, auch die der eingebundenen Blätter) und aktualisiert danach für das aktive Blatt die Symbole und die Testtasten. Nötig ist das zum Beispiel, wenn gespeicherte Blätter aus einer früheren Version stammen und falsche LED-Zahlen in Zeile 1 stehen. Vor dem Erzeugen der Datei `LEDs_AutoProg.h` („Z. Arduino schicken“) werden ohnehin alle Blätter neu gerechnet.
- **Menü Tabelle → Tabelle reparieren (Symbole und Testtasten neu aufbauen) …:** Reparatur, wenn **Symbole oder Testtasten fehlen** oder an der falschen Zeile stehen (zum Beispiel nach einer fehlerhaft geladenen Tabelle). Nach einer Rückfrage werden Symbole und Testtasten **aller Datenblätter** gelöscht und aus dem Tabelleninhalt neu angelegt (auch die Namen aus dem Makro neu gesetzt); das kann bei großen Tabellen einige Sekunden dauern. „Tabelle neu berechnen“ legt fehlende Symbole und Testtasten nicht an. (Die früheren Einträge „Symbole aktualisieren“ und „Testtasten neu aufbauen“ gibt es nicht mehr: Beides macht „Tabelle neu berechnen“ für das aktive Blatt.)

2.10 Der Patterngenerator

Der Reiter **Patterngenerator** erstellt aus einer Tabelle ein Muster für die LEDs (Wechselblinker, Lauflicht usw.). Das Ergebnis ist ein Makro, das im Programmgenerator eingetragen wird. **Stand:** Anzeigen, Bearbeiten, Berechnen sowie **Export zu ProgGen** und **Import von ProgGen** gehen. Dazu kommen **Optionen** mit den Beispielen und die Auswahldialoge für **Spezial Mode**, **RGB Modul Nr** und **Goto Aktivierung**. Die Seiten zeigen auch ihre **Bilder** und die **Goto-Pfeile**, **Bild einfügen** fügt eigene Bilder ein. **Noch nicht** gibt es das Testen auf dem Arduino („Test Pattern“), das Senden an ein Modul (damit auch das Blinken der LED bei RGB Modul Nr), die Wirkung der Spezial Modes auf dem Modul und den Multiplexer.

- **Aufbau:** Oben stehen die gelben Eingabefelder (Erste RGB LED, Anzahl der Ausgabe Kanäle, Mode usw.) und darunter das Ergebnis (`PatternT1(…)`), der Makroname und das Makro. Unten liegt die Tabelle der Dauer pro Spalte und der LEDs. Ein **x** schaltet die LED in dieser Spalte ein, die **Balken** über den Zellen zeigen das Muster. Die Spalten A bis D sind gesperrt.
- **Eingeben:** Zelle anklicken und schreiben, **F2** oder Doppelklick bearbeitet eine Zelle, **Entf** leert sie. **Strg+C**, **Strg+X** und **Strg+V** kopieren, schneiden aus und fügen ein. Beim Einfügen von mehreren Zellen (aus einer Tabellenkalkulation) gilt: Zeilen durch Zeilenwechsel, Spalten durch Tabulator, ab der markierten Zelle. Das Kontextmenü (Rechtsklick) bietet dasselbe. Beim ersten Einfügen aus der Zwischenablage fragt der Browser, ob er sie lesen darf.
- **Bilder:** Die Seiten zeigen ihre Bilder (zum Beispiel das Signal oder die Anlage zum Muster) unter der Tabelle. Die Bilder **bleiben, wo sie sind**: Bilder der Beispiele im Ordner `Pattern_Config_Examples`, eigene Bilder im Datenordner (`pattern_pictures`). Mit der Seite werden nur der Pfad und die Lage gemerkt. Fehlt eine Bilddatei nach einem Neustart, fehlt das Bild.
- **Bild einfügen:** Der Knopf lädt ein PNG- oder JPG-Bild hoch und setzt es in natürlicher Größe unter die Tabelle (Zelle B63). Da das Hochladen nur eine Zwischendatei liefert, wird die Datei in den Datenordner kopiert (gleicher Name und gleicher Inhalt: keine zweite Kopie; gleicher Name und anderer Inhalt: Name mit `_2`). Das Bild lässt sich nicht verschieben. **Optionen → Bilder dieser Seite entfernen** nimmt alle Bilder der Seite weg (die Dateien bleiben). Beim Speichern einer Seite (Optionen) gehören die Bilder zur ZIP-Datei.
- **Aussehen wie in Excel:** Die Kopfzeile der Tabelle (Spalte Nr) ist hellblau, die Spalte LED Nr hellgelb, die Beschriftung der Goto Tabelle hellgrün. Um die Zeile Dauer, die Goto Tabelle und die Tabelle der LEDs (so viele Zeilen, wie LEDs oder Kanäle eingestellt sind) liegen schwarze Rahmen. Die Zeile Ergebnis ist blau, die Zeile Makro grün. Bei aktivem Goto Mode sind die Zeilen über und unter der Goto Tabelle viel höher als die anderen, damit die Pfeile und Bögen Platz haben, wie in Excel.
- **Goto-Pfeile:** Bei aktivem Goto Mode zeigt die Tabelle, wie das Muster startet und springt: grüne Pfeile mit der Nummer der Startspalte, rote Pfeile mit **E** am Ende des Musters und blaue gebogene Pfeile für die Sprünge zwischen den Spalten.
- **Doppelklick auf besondere Zellen:** Ein Doppelklick auf **Spezial Mode** öffnet eine Liste der Spezial Modes. Unter der Liste stehen die ausführliche Beschreibung des gewählten Eintrags; ein Doppelklick auf einen Eintrag oder **Auswahl** trägt ihn in die Zelle ein. Ein Doppelklick auf **RGB Modul Nr** öffnet eine Zahleneingabe mit − und +. Ohne ein Modul am USB-Anschluss blinkt dabei keine LED, die Nummer wird nur eingetragen. Tippen und **F2** bearbeiten die Zelle weiterhin direkt.
- **Goto Aktivierung:** Wenn der Goto Mode benutzt wird, kann das Muster an mehreren Stellen starten (Spalten mit nummerierten Pfeilen). Die **Goto Aktivierung** legt fest, wie die Startspalte im Betrieb gewählt wird (Taster, Zähler, Zufall …). Ein Doppelklick auf die Zelle öffnet die Liste mit Kurz- und ausführlicher Beschreibung, **Löschen** leert die Zelle. Bei aktivem Goto Mode öffnet sich die Liste auch schon, wenn man die **leere** Zelle anklickt. Die Einträge `Counter`, `RandButton`, `RandomTime`, `RandomCount` und `RandomPingPong` fragen danach noch ihre Parameter ab: Zeiten werden als Zahl in Millisekunden oder mit Einheit angegeben (`30 Sek`, `5 Min`, `500 ms`) und auf ihren Bereich geprüft. Beim **Export zu ProgGen** fragt das Programm, wenn der Goto Mode mehrere Startspalten hat und keine Aktivierung eingetragen ist, ob die Auswahl geöffnet werden soll. Dann erscheinen nur Einträge mit genug Ausgängen.
- **Werkzeugleiste:**
  - **Neues Blatt:** legt eine Kopie des aktuellen Blatts an (der Name wird abgefragt, ein Reiter erscheint unten).
  - **Aktualisieren:** zeichnet die Balken neu. **Neu berechnen:** berechnet alle Formeln neu. **Hinweis:** Die Balken werden nicht mitgespeichert. Nach einem Neustart des Programms zeichnet jede Seite ihre Balken beim ersten Anzeigen (Reiter anklicken) neu. Fehlen sie trotzdem, hilft **Aktualisieren**. Die Tabelle selbst ist vollständig wieder da, und die **Bilder** der Seiten kommen von selbst zurück.
  - **+** und **−:** fügt vor der markierten Zelle eine Spalte in die Tabelle ein bzw. löscht die Spalte. Dazu muss die markierte Zelle **in der Tabelle** liegen (LED-Tabelle, Dauer oder Goto-Zeile), sonst erscheint die Meldung „Cursor outside the table“. Beim ersten Mal erklärt das Programm die Funktion.

    - **RGB LED:** schaltet das Blatt auf RGB-LEDs um.
    - **Optionen:** öffnet einen Dialog mit den Beispielen und der Dateiverwaltung.
        - **Beispiele laden:** Haken bei den gewünschten Sammlungen (zum Beispiel „Ampeln“, „Signale“, „Baustellenlichter“) oder „Alle Beispiele“, dann **Ausgewählte Beispiele laden**. Jede Sammlung bringt mehrere Seiten mit, sie erscheinen als neue Reiter. Seiten, die es schon gibt, werden nicht noch einmal geladen.
        - **Datei laden …:** lädt eine eigene Datei (`.MLL_pcf`) oder eine ZIP-Datei mit der Datei und ihren Bildern. Sie wird über den Browser hochgeladen.
        - **Aktuelle Seite speichern**, **Alle Seiten speichern:** schreibt die Seite(n) in eine Datei im Format des Excel-Programms (`.MLL_pcf`) und bietet sie zum Speichern an (Speichern unter … oder Downloads-Ordner). Gehören Bilder zu den Seiten, entsteht eine ZIP-Datei mit allen Dateien.
        - **Aktuelle Seite löschen**, **Alle Beispiele löschen:** löscht die Seite bzw. alle Seiten außer Main nach einer Rückfrage. Das kann nicht rückgängig gemacht werden.
    - **Export zu ProgGen:** trägt das Makro des Blatts in eine Zeile des Programm Generators ein. Vorher prüft das Programm, ob ein Makroname eingetragen ist, die LED-Tabelle nicht leer ist und die Dauer-Zeile stimmt, und meldet sonst den Fehler. Danach springt der Browser zum Programm Generator, über der Tabelle erscheint ein **gelbes Band**: In die Zeile klicken, in der das Muster stehen soll, und **OK** drücken. Die Haken „gleich zum Arduino schicken“ (erzeugt die Dateien, kompiliert und lädt hoch) und „Anschließend zurück zum Patterngenerator“ gelten für diesen Vorgang. Bereits vorhandene Einträge in der Spalte „Beleuchtung, Sound, oder andere Effekte“ werden überschrieben, Symbol und Name der Zeile kommen dazu. **Abbrechen** bringt zurück zum Patterngenerator.
    - **Import von ProgGen:** liest ein Muster aus einer Zeile des Programm Generators. Im gelben Band eine Zeile mit einem `PatternT…`-Makro anklicken und **OK** drücken. Das Programm fragt, ob die Daten in das aktuelle Blatt kommen oder ein neues Blatt angelegt wird. Danach zeigt der Browser den Patterngenerator. Wie in der Excel-Version steht im Feld „Schalter Nummer“ danach `#InCh`.
- **Beschreibung:** Der Text des Blatts lässt sich über „Beschreibung“ oben aufklappen.
- **Spaltenbreite:** Die Muster-Spalten sind gleich breit. Das Programm passt die benutzten Spalten an die Dauer an, wenn die Zeiten sehr unterschiedlich sind (wie in Excel). In dieser Tabelle lassen sich die Spalten nicht von Hand verstellen, weil die Balken an den Spaltenbreiten hängen.
- **Daten:** Die Blätter werden beim Beenden im Datenordner gesichert (`temp_workbook.table_PatternWorkbook.json`) und beim nächsten Start geladen.

Dateien aus dem Excel-Programm laden: Dateien im Format `.MLL_pcf` und `.MLL_pgf`, die mit dem Excel-Programm gespeichert wurden, haben den Windows-Zeichensatz (cp1252; als Trennzeichen im Patterngenerator dient das Zeichen `-`). Das Programm erkennt den Zeichensatz beim Laden selbst: Dateien in **UTF-8** (vom Web-Programm gespeichert, auch mit Kennzeichen am Anfang) und im **Windows-Zeichensatz** lassen sich beide laden, mit Umlauten und Sonderzeichen. Früher kam bei einer Datei aus Excel die Meldung „‘utf-8’ codec can’t decode byte 0xaf …“. Beim **Speichern** schreibt das Programm die Dateien (`.MLL_pcf` und `.MLL_pgf`) ebenfalls im **Windows-Zeichensatz**, damit auch das Excel-Programm sie lesen kann (sonst stimmen Trennzeichen und Umlaute dort nicht). Das lässt sich in den **Programm Einstellungen** mit der Option **Dateien für das Excel-Programm speichern** abschalten (Standard: an). Enthält eine Datei Zeichen, die im Windows-Zeichensatz keinen Platz haben (zum Beispiel ein polnisches „ł“), bleibt sie UTF-8; das Excel-Programm kann sie dann nicht richtig lesen. Das Laden geht immer mit beiden.

2.11 Vollbild und Adressen der Reiter

- **Verbindung zum Arduino:** In der Kopfzeile, links neben **Vollbild**, steht für **alle Reiter** der Knopf **Mit ARDUINO verbinden**. Er ist **rot**, solange keine Verbindung besteht, und **grün** („Mit ARDUINO verbunden“), wenn die Verbindung steht (auch wenn man im Menü ARDUINO → Verbinden oder Trennen wählt). Ein Klick auf den grünen Knopf bewirkt nichts; getrennt wird im Menü ARDUINO → Trennen. Nach dem Verbinden startet ein offener Reiter mit Direktmodus (Servo Einstellungen, DM Servo Einstellungen, Sound Test) den Direktmodus neu.
- **Version der Bibliothek:** Auf der Seite Programm Generator steht rechts in der Zeile der Funktionsknöpfe die Version der MobaLedLib, zum Beispiel **Ver.3.5.0-K** (wie in Excel). Das Feld bleibt leer, solange die Bibliothek nicht installiert ist. Die Version wird beim Aufbau der Seite, nach der Prüfung der Bibliotheken beim Start und nach jedem Befehl (zum Beispiel einer Installation der Bibliotheken) gelesen, nicht laufend.
- **Vollbild:** Der Knopf **Vollbild** in der Kopfzeile (neben **Beenden**) blendet Titel, Menü und Reiterleiste aus. Die Statusmeldungen und die Knöpfe Vollbild und Beenden bleiben, die Tabelle bekommt den Platz. Wenn der

Browser es erlaubt, geht er dabei auch in den Vollbildmodus (auf dem iPhone gibt es das nicht, dort wirkt nur das Ausblenden). Derselbe Knopf (dann **Vollbild beenden**) oder die Taste ESC schaltet zurück. Gedacht ist es für kleine Bildschirme, zum Beispiel das Smartphone. Im Vollbild ist die Reiterleiste weg: Zum Wechseln erst das Vollbild beenden.

- **Reiter direkt aufrufen:** Jeder Reiter hat eine eigene Adresse, zum Beispiel `http://localhost:8080/Farbtest`:

Adresse	Reiter
`/ProgGenerator`	Programm Generator
`/Patterngenerator`	Patterngenerator
`/ProgGeneratorEinstellungen`	ProgGenerator Einstellungen
`/Farbtest`	Farb Test
`/ServoTest1`	Servo Einstellungen
`/ServoTest2`	DM Servo Einstellungen (nur mit eingeschalteter Option, siehe 2.13)
`/SoundTest`	Sound Test
`/DCCKeyboard`	DCC Keyboard
`/Z21Simulator`	Z21 Simulator (nur mit eingeschalteter Option, siehe 2.14)
`/SerialMonitor`	Serial Monitor
`/ARDUINOEinstellungen`	ARDUINO Einstellungen
`/ProgrammEinstellungen`	Programm Einstellungen

Groß- und Kleinschreibung ist egal (`/farbtest` wird umgeleitet). Mit dem Zusatz `?vollbild=1` öffnet sich der Reiter gleich im Vollbild (zum Beispiel `/ServoTest1?vollbild=1`); der erste Tipp auf die Seite schaltet dann auch den Browser auf Vollbild. Bei einem unbekannten Namen zeigt das Programm die Liste der gültigen Adressen. Die Startseite `/` bleibt der Programm Generator. Aus dem Netz heißt es `http://<Adresse des Rechners>:8080/Farbtest`; das Passwort (Kapitel 8) gilt auch hier.

2.12 Servo Einstellungen

Der Reiter **Servo Einstellungen** (früher Servo Test 1, Adresse `/ServoTest1`) bedient die Servoplatine der MobaLedLib (Servos über LED-Befehle) und ist wie der **Excel-Dialog „Servo Test und Einstellung“** aufgebaut. Er arbeitet über dieselbe serielle Verbindung wie der Farb Test: zuerst mit dem Arduino **verbinden** (Knopf **Mit ARDUINO verbinden** in der Kopfzeile oder Menü ARDUINO → Verbinden).

- **Servo wählen:** In der Zeile **Servo Modul Auswahl** stehen **LED Nummer**, **Anschluss** (0 bis 2) und **LED Kanal**. Nach jeder Änderung wählt das Programm den Servo und lässt ihn **kurz wackeln**, damit man ihn erkennt. Die Werte werden gemerkt.
- **Position:** Der Regler (10 bis 210) mit den Pfeilen daneben stellt die Position ein, rechts steht der Wert.
- **Servo Konfigurieren** führt in drei Schritten durch **Minimalposition**, **Maximalposition** und **Geschwindigkeit**. Mit den Pfeiltasten **<< < > >>** wird der Wert eingestellt (**gedrückt halten**: schneller). **Weiter** geht zum nächsten Schritt, **Einstellungen speichern** speichert (in jedem Schritt möglich), **nicht speichern** bricht ab.
- **Servo zurücksetzen** stellt nach einer Rückfrage die Grundeinstellungen des Servos wieder her (Limits und Geschwindigkeit).
- **Beschreibung:** Der Text oben ist einzeilig; ein Tipp darauf klappt ihn auf.
- **Verlassen:** Beim Wechseln des Reiters, beim Schließen oder Neuladen der Seite und beim Beenden des Programms bekommt der Servo den **Wert 0**, ein laufender Ablauf wird abgebrochen und der Direktmodus beendet. So bleibt kein Servo in Bewegung.
- Die Seite ist für das **Smartphone** gebaut (eine Spalte, große Tasten); zusammen mit dem Vollbild (2.11) bleibt viel Platz.

2.13 DM Servo Einstellungen, Sound Test und DCC Keyboard

- **DM Servo Einstellungen** (früher Servo Test 2, Adresse `/ServoTest2`) steuert Servos im **Direct Mode** (ein ATtiny statt des WS2811-Chips). Parameter: **Servo Modul Adresse**, **LED Kanal**, **Servo Betriebsart** (1 Normal, 2 und 3 Training, 4 Max Speed, 5 Invers, 6 Servo zurücksetzen, 7 Werkseinstellung, 8 letzte Position zurücksetzen, 9 Oszillator trimmen, 10 Speed Legacy; 0: es wird nichts gesendet) und **Fast Refresh**. Regler 0 bis 255, **Pos 0**, **Pos 255** und die Taste **ENTER** (gedrückt halten; in den Betriebsarten 2 und 3 zweimal

drücken, die Beschriftung zeigt „2nd Step“). Nach ENTER in den Betriebsarten 4 bis 8 und 10 steht die Betriebsart auf 0. Die Betriebsart wird nicht gemerkt, die Seite beginnt immer mit 1. Das Programmieren des ATtiny (Hex-Datei) und das Formular für den Programmieradapter gibt es noch nicht. **Der Reiter erscheint nur, wenn in den Programm Einstellungen die Option „DM Servo Einstellungen anzeigen“ eingeschaltet ist** (und nach dem Speichern sofort, ohne Neustart; vorher hieß die Option „Programmgenerator anzeigen“ und hatte im Web keine Wirkung). Ruft man `/ServoTest2` bei ausgeschalteter Option auf, kommt ein Hinweis, und der Programm Generator bleibt offen.

- **Sound Test** (`/SoundTest`) prüft die MLL-Soundplatine (MP3-TF-16): **Soundmodul-Adresse**, **LED Kanal**, **Impulslänge** und die Werte **R, G, B** (Sound-Auswahl) bestimmen den Ton. **Sound an ARDUINO schicken** sendet den Wert und beendet den Impuls nach der Impulslänge; Änderungen an Adresse, Impulslänge, R, G und B senden gleich (nach kurzer Ruhe beim Tippen). Zum Verbinden dient der Knopf **Mit ARDUINO verbinden** in der Kopfzeile (siehe 2.11). Beim Verlassen des Reiters oder der Seite wird ein laufender Impuls beendet. Die Tastenkürzel Strg-R und Alt-R der Tk-Seite gibt es nicht (Strg-R lädt im Browser die Seite neu).
- **DCC Keyboard** (`/DCCKeyboard`) schickt dem ARDUINO die Kommandos `@ AAA CC 01`, die sonst der DCC-ARDUINO an den LED-ARDUINO schickt, damit man die DCC-Schalter ausprobieren kann: 32 Adressen ab der **DCC-Basisadresse**, je ein roter (Kanal 0, mit der Adresse beschriftet) und ein grüner Knopf (Kanal 1), in vier Gruppen zu acht. Beim ersten Tastendruck verbindet die Seite selbst; das kann etwas dauern, und manchmal reagiert der ARDUINO erst auf den zweiten Tastendruck.
- Alle drei Seiten sind für das Smartphone gebaut (Beschreibung einzeilig und aufklappbar, große Tasten) und im Vollbild (2.11) gut zu bedienen.

2.14 Z21 Simulator

Der Reiter **Z21 Simulator** (`/Z21Simulator`) gibt sich gegenüber Modellbahn-Steuerprogrammen (Rocrail, iTrain, Traincontroller, WinDigiPet, Z21-Apps) als **Roco Z21** aus. Weichen- und Lokbefehle des Steuerprogramms schickt er als DCC-Kommando an den Arduino (`@ AAA CC AA`), so lassen sich die DCC-Schalter der MobaLedLib direkt aus dem Steuerprogramm bedienen. Die Z21 zählt Adressen ab 0, die MobaLedLib ab 1; der Simulator rechnet das um.

- **Einschalten:** Programm Einstellungen → **Z21 Simulator anzeigen**, dann Speichern: Der Reiter erscheint sofort (ohne Neustart).
- **Bedienung:** Zuerst mit dem Arduino verbinden (Knopf **Mit ARDUINO verbinden** in der Kopfzeile; der Start versucht es selbst, startet aber auch ohne Verbindung). Dann **Start Simulator**. Die Seite zeigt die **IP-Adresse und den Port 21105 (UDP)**, die im Steuerprogramm als Z21 einzutragen sind. Läuft das Steuerprogramm auf demselben Rechner, genügt `127.0.0.1`. Im Fenster darunter steht jedes Paket mit Uhrzeit, Absender und dem Befehl an den Arduino. **Stop Simulator** beendet ihn. **Zeige Taster-Adressen** zeigt die Eingangskanäle aus der zuletzt erzeugten Header-Datei.
- Der Simulator läuft weiter, wenn man den Reiter wechselt oder die Seite neu lädt. Er endet mit **Stop** oder mit dem Beenden des Programms.
- **Firewall und Netz:** Das Steuerprogramm muss den **UDP-Port 21105** erreichen. **Windows** fragt beim ersten Start nach der Freigabe für Python: **Private Netzwerke** erlauben. **Linux** mit `ufw`: `sudo ufw allow 21105/udp`. Auf dem **Raspberry Pi** ist der Port normalerweise frei, der Simulator läuft dann dauerhaft auf dem Pi. Ist der Port schon belegt (andere Z21-Software, zweiter Simulator), erscheint eine Meldung.
- **Rückmeldung (RMBUS):** Hat das Steuerprogramm die Rückmeldung angemeldet (Broadcast-Flag „R-Bus“, so tun es zum Beispiel Rocrail und iTrain für die Rückmelder), fragt der Simulator den Arduino alle 100 ms nach dem Stand der Taster und Eingänge und meldet **Änderungen** an das Steuerprogramm (je Gruppe von 10 Modulen). Im Fenster erscheint nur eine Zeile, wenn sich etwas ändert; die Statuszeile nennt, für wie viele Programme die Rückmeldung läuft. Das Intervall steht in der (versteckten) Einstellung `RMbusTimer` (× 100 ms; 0 oder leer = 100 ms).
- **Noch nicht umgesetzt:** Die **Port-Liste** (bis zu drei Arduinos mit eigenen DCC-Adressbereichen); es wird der eine mit dem Programm verbundene Arduino benutzt.

2.15 Sprache der Oberfläche

In den **Programm Einstellungen** wählt man unter **Sprache**: Auto (die Sprache des Rechners, auf dem das Programm läuft), Deutsch, English, Nederlands, French, Italian, Spanish oder Denmark. Nach dem **Speichern** wirkt die Auswahl **sofort**: Die Seite wird in allen Browsern neu geladen, und alle Texte erscheinen in der neuen Sprache. Ein Neustart ist nicht nötig.

- **Was übersetzt wird:** alle Texte der Oberfläche: Menüs, Reiter, Knöpfe, Hinweise, die Texte und

Beschreibungen in den Formularen (zum Beispiel „Verteiler und Stecker Nummer“, „Optionen und spezielle Funktionen“) und in den **Programm Einstellungen**, die Meldungen des Programm Generators und des Patterngenerators, die **Spaltenköpfe** mit ihren Hinweisen, die **Namensspalte** und die **Typen** in der Tabelle. Die Texte stehen an zwei Stellen: Die Texte, die es schon im Excel-Programm gab, kommen wie dort aus der Tabelle Languages der Excel-Datei; alles, was es nur im Web-Programm gibt, steht im Katalog `python/csv/Web_Languages.csv`. Fehlt ein Text in der Tabelle Languages, wird im Katalog nachgesehen. Stand: **English** ist vollständig (Entwurf), die anderen Sprachen sind nur dort übersetzt, wo die Tabelle Languages eine Übersetzung hat (sonst erscheint English).

- **Gespeichert wird immer deutsch:** In den Zellen, in den Dateien (`.MLL_pgf`) und im erzeugten Arduino-Programm stehen die deutschen Begriffe. Ein Sprachwechsel **ändert keine Daten**; Dateien lassen sich auch in einer anderen Sprache lesen. Eine Datei mit den Begriffen einer anderen Sprache (zum Beispiel aus dem Excel-Programm mit niederländischer Oberfläche) wird beim Laden in die deutschen Begriffe umgewandelt. In der Spalte Typ darf man die Begriffe oder ihren ersten Buchstaben in der eingestellten Sprache eintippen.
- **Spaltenköpfe und Namensspalte:** Die Köpfe der Datenblätter (Zeile 2, mit ihrem Hinweistext beim Daraufzeigen) und die **Namensspalte** (der Name des Makros, zum Beispiel „Belebtes Haus“ / „Busy house“ / „Druk huis“) erscheinen in der gewählten Sprache. Auch das nur in der Anzeige: Die Zellen bleiben unverändert. Ein Kopf, den man selbst geändert hat, bleibt, wie er ist. Der Name wird wie in Excel aus der Tabelle Lib_Macros bestimmt (Spalte Name der Sprache, sonst der Name ohne Sprache), auch auf Deutsch: Ein Name, der beim Laden einer Datei in einer anderen Sprache entstanden ist, wird dadurch ersetzt.
- **Meldungen mit eingesetzten Werten und zusammengesetzte Meldungen** (zum Beispiel „Fehler beim Speichern: …“, die Meldungen des Updates, der Firewall, des Z21 Simulators, „… Seite(n) gelöscht“) werden ebenfalls übersetzt; die eingesetzten Werte (Dateinamen, Zahlen, Fehlertexte des Systems) bleiben, wie sie sind.
- **Noch nicht übersetzt** (erscheinen auf Deutsch): die Beschreibungen in den Beispielblättern, die Anleitung und Texte, die nur im Excel-Programm vorkommen (Animationsformulare der Tk-Version).
- **Fehlende Übersetzungen:** Fehlt ein Text in der gewählten Sprache, erscheint der englische, fehlt auch der, der deutsche. Stand: English (Entwurf) für alle Texte der Oberfläche (766 Einträge im Katalog; English, Nederlands, Français, Italiano, Español und Dansk als Entwürfe); Nederlands, French, Italian, Spanish und Denmark folgen. Wie man mitübersetzt, steht in `install/UEBERSETZEN.md`.

2.16 Entwicklungsumgebung: ARDUINO IDE oder PlatformIO

Programme werden mit der **ARDUINO IDE** (Standard, Abschnitt 2.2) oder mit **PlatformIO** übersetzt und auf den Arduino, Pico oder ESP32 geladen. PlatformIO ist ein Ersatz für die ARDUINO IDE; man braucht dafür nur das Kommandozeilenprogramm `pio`, nicht die IDE.

- **Umschalten:** Menü **Optionen → Optionen und spezielle Funktionen**, Reiter **Entwicklungsumgebung**: *ARDUINO-IDE (Standard)* oder *PlatformIO (Ersatz für die ARDUINO-IDE)*. Die Auswahl wird beim Schließen des Dialogs übernommen. Darunter steht, ob PlatformIO gefunden wurde; der Knopf **PlatformIO prüfen** zeigt die Version.
- **Installieren:** PlatformIO muss einmal installiert werden (Anleitung: https://platformio.org/install/cli).
  - **Windows:** zum Beispiel mit `pip install platformio` oder mit der PlatformIO-Erweiterung von Visual Studio Code.
  - **Linux, macOS und Raspberry Pi:** mit **pipx**: `pipx install platformio`, danach `pipx ensurepath` und neu anmelden (oder den Rechner neu starten). Debian, Ubuntu und Raspberry Pi OS erlauben `pip install` nicht außerhalb einer virtuellen Umgebung (Meldung „externally-managed-environment“); dort vorher `sudo apt install pipx`. Das Installationsskript von platformio.org (Ordner `~/.platformio`) geht ebenfalls.
  - **Mit dem Installationsskript:** `install_windows.bat /platformio`, `bash install_linux.sh --with-platformio`, `bash install_mac.command --with-platformio` oder `sudo bash install_pi.sh --with-platformio` (auch zusammen mit `--with-arduino-ide`). Das Skript legt eine **eigene Python-Umgebung** `~/.platformio/penv` an (Windows: `%USERPROFILE%\.platformio\penv`), genau wie der Installer von platformio.org, und installiert PlatformIO dort hinein. Das geht auch auf Debian, Ubuntu und Raspberry Pi OS (die Meldung „externally-managed-environment“ tritt nicht auf), `pipx` ist nicht nötig. Auf Linux, macOS und Raspberry Pi geht es auch einzeln und nachträglich: `bash install/install_platformio.sh` (ohne `sudo`). Auf dem Raspberry Pi gehört die Umgebung dem Benutzer des Dienstes. Das Skript installiert nur PlatformIO; danach in den Optionen **Entwicklungsumgebung** auf PlatformIO einstellen (**PlatformIO prüfen** zeigt den Stand). Das erste Hochladen lädt die Compiler und Bibliotheken nach (Internet): **etwa 5 Minuten beim Nano und etwa 10 Minuten beim Pico**, nur beim ersten Mal. Die Installation von PlatformIO selbst lädt ebenfalls eine größere Datenmenge.

    - Das Programm findet `pio` selbst: im Ordner von PlatformIO (**Windows:** `%USERPROFILE%\.platformio\penv\Scripts`; **Linux, macOS und Raspberry Pi:** `~/.platformio/penv/bin`), unter Linux und macOS auch in `~/.local/bin` (dort legt pipx es ab), und im Pfad (PATH). Fehlt es, erscheint beim Hochladen eine Meldung mit dem Installationshinweis; man kann dann in den Optionen wieder die ARDUINO IDE einstellen.
- **Ablauf:** Beim Hochladen schreibt das Programm die Datei `platformio.ini` (im Ordner `python`) mit den aktuellen Einstellungen (Boards `nano_new`, `nano_old`, `nano_full`, `pico`, `esp32`; die Bibliothek MobaLedLib kommt aus dem Ordner `libraries`) und startet `pio run -t upload`. Beim ersten Mal lädt PlatformIO die Werkzeuge für das Board aus dem Internet; das kann einige Minuten dauern.
- **Raspberry Pi Pico unter Windows:** Für das Hochladen mit PlatformIO braucht der Pico unter Windows **einmalig einen zusätzlichen USB-Treiber**. Den Pico mit gedrückter BOOTSEL-Taste anstecken (Bootloader-Modus), das Programm **Zadig** starten (https://zadig.akeo.ie), unter *Options* → *List All Devices* den Eintrag **„RP2 Boot (Interface 1)“** wählen, als Treiber **WinUSB** einstellen und *Install Driver* (oder *Replace Driver*) anklicken. Danach lässt sich der Pico hochladen. Fehlt der Treiber, steht in der Fehlermeldung von PlatformIO bereits ein Hinweis auf Zadig.
- **Raspberry Pi Pico unter Linux:** Linux erlaubt einem normalen Benutzer den Zugriff auf den Pico im Bootloader-Modus nur mit einer **udev-Regel**. Fehlt sie, meldet das Hochladen: „RP2040 device … appears to be in BOOTSEL mode, but picotool was unable to connect. Maybe try ‘sudo’ or check your permissions“. Das Programm weist vor dem Hochladen darauf hin. Einrichten (einmalig): **Optionen → Optionen und spezielle Funktionen → Entwicklungsumgebung → „USB-Zugriff für den Pico einrichten“** (fragt nach dem Passwort) oder im Terminal `sudo bash install/install_pico_udev.sh`. Danach den Pico neu anstecken (BOOTSEL-Taste gedrückt halten). Das Skript legt die Regel `/etc/udev/rules.d/99-mobaledlib-pico.rules` an und nimmt den Benutzer in die Gruppen `plugdev` und `dialout` auf (gilt nach dem nächsten Anmelden).
- **PlatformIO ganz ohne ARDUINO IDE:** Ist **PlatformIO** als Entwicklungsumgebung eingestellt (Optionen → Entwicklungsumgebung), braucht das Programm die ARDUINO IDE **nicht**. Dann gilt:
    - Beim Start gibt es **keine Suche nach der IDE** (keine Meldung „ARDUINO IDE nicht gefunden“) und **keine Prüfung oder Aktualisierung der ARDUINO Bibliotheken und Boards**.
    - Der **Sketchbook-Pfad ist der Python-Ordner**: Das Programm liest die Datei `preferences.txt` der IDE nicht mehr. Der Ordner `libraries` im Python-Ordner enthält die MobaLedLib (für PlatformIO) und die Erweiterungen (Extensions). Die Version der MobaLedLib steht in `libraries/MobaLedLib/library.properties`. FastLED, NmraDcc und weitere Bibliotheken lädt PlatformIO beim ersten Übersetzen selbst.
    - Die Reiter **Update** (Bibliotheken und Boards der IDE aktualisieren) und **Bootloader** (schneller Bootloader, braucht ArduinoISP und avrdude der IDE; bei einem WLAN-Pico bleibt der Reiter) werden ausgeblendet; sie erscheinen wieder, wenn man die ARDUINO IDE einstellt. Das Update des Programms selbst bleibt im Menü **Hilfe → Update webMobaLedLib**.
    - **ESP32** und **Pico** sind als Board wählbar, auch wenn ihre Pakete in der IDE fehlen.
    - Schaltet man auf die ARDUINO IDE zurück, fragt das Programm, ob die Bibliotheken und Boards jetzt geprüft werden sollen (und meldet, wenn die IDE nicht gefunden wird).
    - Weiter an die IDE gebunden sind der schnelle Bootloader und das Übersetzen des Simulators (nur Windows).
- **Ordner `.pio`:** PlatformIO legt im Ordner `python` den Ordner `.pio` an (Übersetzungsergebnisse und geladene Bibliotheken, **mehrere hundert MB**). Er gehört nicht ins Release (`zip_erstellen.bat` lässt ihn weg), Update und Reparatur fassen ihn nicht an. Man kann ihn jederzeit löschen, um Platz zu schaffen; beim nächsten Hochladen wird er neu aufgebaut (das dauert dann länger).
- **Hinweis:** PlatformIO ist neu im Programm. Rückmeldungen, vor allem von Linux, macOS und dem Raspberry Pi, sind willkommen.

2.17 Anzahl der Eingänge (Excel-Programm 3.5.0-K-long)

Das Programm beruht auf dem Excel-Programm **3.5.0-K-long** der MobaLedLib. Es erlaubt je nach Board unterschiedlich viele Eingänge (Variablen): **Arduino Nano und Uno bis 250**, **ESP32 und Pico bis 2048** (Tabelle Platform_Parameters, Zeile MaxInput). Sind es zu viele, meldet das Programm beim Erzeugen „Die Anzahl der verwendeten Eingangskanäle ist zu groß“ mit der erlaubten Zahl. Dazu gehört die Bibliothek MobaLedLib in der Version **3.5.0-K-long** (Optionen → Optionen und spezielle Funktionen → Update); ein beim Laden von GitHub entstandener Ordner mit dem Zusatz `-longInpAddresses` wird dabei automatisch umbenannt.

3. Windows

Voraussetzungen: Windows 10 oder 11, Python 3.10 oder neuer (das Skript bietet die Installation an), Internet für die Installation der Python-Pakete.

Installation

1. Im Ordner `install` **`install_windows.bat` doppelklicken**. Für alles in einem Zug (IDE und Bibliotheken) in einer Eingabeaufforderung im Ordner `install` `install_windows.bat /ide` eingeben. Meldet Windows „Der Computer wurde durch Windows geschützt“: *Weitere Informationen* → *Trotzdem ausführen*.
2. Das Skript sucht Python 3.10 oder neuer. Fehlt es, bietet es die Installation von Python 3.12 mit `winget` an und sucht danach in den Standardordnern weiter, die Installation läuft dann ohne Pause weiter. Findet es Python danach nicht, meldet es das: **Dann den PC neu starten** (oder ab- und wieder anmelden) und das Skript noch einmal starten. Erst nach dem Neustart kennt Windows den neuen Suchpfad von Python.
3. Es legt die Python-Umgebung `venv_web` im Projektordner an, installiert die Pakete aus `requirements-web.txt`, erzeugt das Programmsymbol und legt auf dem Desktop die Verknüpfung **„webMobaLedLib“** an.
4. Mit `/ide` lädt es die IDE 1.8.19 (ca. 130 MB) von arduino.cc und führt den Installer still aus. Windows fragt nach der Erlaubnis (Benutzerkontensteuerung); den Treiber für den USB-Anschluss bitte zulassen. Danach installiert es FastLED und MobaLedLib. (Nur die Bibliotheken, wenn die IDE schon da ist: `/libs`.)
5. Am Ende läuft ein Test der Installation (`pi_check.py`, etwa eine Minute). Ohne ihn: `/nocheck`.

Option	Wirkung
`/ide`	ARDUINO IDE 1.8.19 laden und installieren, dazu `/libs`
`/libs`	Bibliotheken FastLED (3.9.12) und MobaLedLib mit `arduino_debug.exe` installieren
`/platformio`	PlatformIO (Ersatz für die ARDUINO IDE) in eine eigene Python-Umgebung `%USERPROFILE%\.platformio\penv` installieren (siehe 2.16)
`/nocheck`	ohne den Test der Installation am Ende

Starten: Verknüpfung auf dem Desktop doppelklicken (oder `install\start_windows.bat`). Es öffnet sich ein (minimiertes) Konsolenfenster und nach 20 bis 60 Sekunden der Browser. Das Konsolenfenster nicht schließen. **Beenden:** Knopf „Beenden“ in der Oberfläche.

Besonderheiten: Der Pfad des Projektordners sollte kein `'` und kein `&` enthalten (sonst kann die Verknüpfung nicht angelegt werden; dann `install\start_windows.bat` per Rechtsklick → *Senden an* → *Desktop*). Die Windows-Firewall fragt nicht, weil die Oberfläche nur auf dem eigenen Rechner lauscht (für den Zugriff aus dem Netz siehe Kapitel 8).

4. macOS

Voraussetzungen: macOS 10.13 oder neuer, Python 3.10 oder neuer (das von macOS mitgelieferte Python ist zu alt), Internet. **macOS 10.11 und 10.12 gehen nicht:** Für diese Systeme gibt es keine fertigen Pakete (zum Beispiel `orjson`), die Installation der Python-Pakete scheitert.

Getestet: MacBook Air (2024) mit Apple M3, 8 GB RAM und macOS 27.0 („Golden Gate“) mit Version 1.38 des Programms: Der Tester meldet, dass es funktioniert.

Installation

1. Im Ordner `install` **`install_mac.command` doppelklicken**. Ist es nicht ausführbar, im Terminal `chmod +x install/install_mac.command` und dann `bash install/install_mac.command`. Für alles in einem Zug: `bash install_mac.command --with-arduino-ide`.
2. Das Skript sucht Python 3.10 oder neuer und bietet bei fehlendem Python die Installation mit Homebrew an. Sonst: Python 3.12 von python.org installieren und das Skript noch einmal starten.
3. Es legt `venv_web` im Projektordner an, installiert die Pakete, erzeugt das Symbol und legt auf dem Schreibtisch das Programm **`webMobaLedLib.app`** an.
4. Mit `--with-arduino-ide` lädt es `arduino-1.8.19-macosx.zip` (ca. 200 MB) und kopiert `Arduino.app` nach *Programme*, danach installiert es FastLED und MobaLedLib. Auf Macs mit Apple-Chip fragt macOS beim ersten Start der IDE nach Rosetta: bitte zustimmen. (Nur die Bibliotheken: `--with-arduino-libs`.)
5. Am Ende läuft ein Test der Installation (ohne ihn: `--nocheck`).

Option	Wirkung
`--with-arduino-ide`	ARDUINO IDE 1.8.19 laden und installieren, dazu `--with-arduino-libs`
`--with-arduino-libs`	Bibliotheken FastLED (3.9.12) und MobaLedLib mit der IDE installieren
`--with-platformio`	PlatformIO in eine eigene Python-Umgebung `~/.platformio/penv` installieren (siehe 2.16)
`--nocheck`	ohne den Test der Installation am Ende

Starten: `webMobaLedLib` auf dem Schreibtisch doppelklicken. Beim ersten Mal fragt macOS eventuell: Rechtsklick → *Öffnen* → *Öffnen*. Die Oberfläche startet ohne Terminalfenster und öffnet den Browser. Meldungen: `webmobaledlib-console.log` und `webmobaledlib.log` im Datenordner (2.6). Zum Beobachten im Terminal: `install/start_mac.command`. **Beenden:** Knopf „Beenden“ in der Oberfläche.

5. Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)

Das Programm läuft im Benutzerkonto, ohne Systemdienst. Du startest es mit einem Symbol, die Oberfläche öffnet sich im Browser.

Voraussetzungen: Debian 12, Linux Mint 21 / 22, Ubuntu 22.04 oder neuer (Fedora, Arch, openSUSE gehen auch, die Paketnamen setzt das Skript). **Python 3.10 oder neuer** (Mint 20 und Debian 11 sind zu alt). Ein Benutzer mit `sudo` (nur für die Systempakete und die Gruppe `dialout`). Internet.

Installation (als normaler Benutzer, **ohne** sudo):

```
cd ~/mobaledlib/python/install
bash install_linux.sh --with-arduino-ide
```

Das Skript erledigt der Reihe nach:

1. Systempakete (`python3`, `python3-venv`, `python3-pip`, `curl`, `xz-utils`), nur was fehlt, mit einer Passwortabfrage von `sudo`.
2. Prüfung auf Python 3.10 oder neuer.
3. Die Python-Umgebung `venv_web` im Projektordner mit den Paketen aus `requirements-web.txt`.
4. Der Benutzer kommt in die Gruppe **`dialout`** (Zugriff auf `/dev/ttyUSB0`, `/dev/ttyACM0`). **Danach einmal ab- und wieder anmelden.**
5. Mit `--with-arduino-ide`: die IDE 1.8.19 nach `~/arduino-1.8.19` (etwa 150 MB) und danach FastLED und MobaLedLib.
6. Ein Eintrag im Anwendungsmenü (Entwicklung / Elektronik) und ein Symbol auf dem Schreibtisch.
7. Ein Test der Installation (`pi_check.py`, etwa eine Minute).

Option	Wirkung
`--with-arduino-ide`	IDE 1.8.19 laden, dazu `--with-arduino-libs`
`--with-arduino-libs`	nur die Bibliotheken (die IDE ist schon da)
`--with-platformio`	PlatformIO in eine eigene Python-Umgebung `~/.platformio/penv` installieren (siehe 2.16)
`--no-desktop-icon`	kein Symbol auf dem Schreibtisch
`--no-dialout`	die Gruppe `dialout` nicht eintragen
`--no-check`	ohne den Test der Installation

Das Skript kann **mehrfach** laufen (zum Beispiel nach einem Update).

Starten: Symbol „webMobaLedLib“ doppelklicken (Schreibtisch oder Anwendungsmenü), oder `install/start_linux.sh`. Beim ersten Doppelklick fragt der Dateimanager (Nemo, Nautilus, Dolphin) eventuell **„Ausführen“** bzw. **„Starter vertrauen“**; sonst Rechtsklick → *Ausführen erlauben* (Mint: in den Eigenschaften *Als Programm ausführen*). **Beenden:** Knopf „Beenden“ in der Oberfläche. Zum Beobachten der Meldungen `install/start_linux.sh` im Terminal starten. Auf dem Desktop läuft die IDE mit deinem Bildschirm, es wird kein virtuelles Display gebraucht.

6. Raspberry Pi (Dauerdienst)

Der Programmgenerator läuft als Dienst auf dem Pi. Du startest ihn mit einem **Desktop-Symbol** oder öffnest ihn mit dem Browser, auf dem Pi selbst oder von jedem Rechner im Netz. Der Arduino/Pico steckt per USB am Pi. Es wird kein Tk gebraucht.

Voraussetzungen: Raspberry Pi 4 oder 5 (ein Pi 3 läuft, Kompilieren dauert dort deutlich länger); Raspberry Pi OS **64 Bit**, **Trixie** (Debian 13, Python 3.13) oder Bookworm, mit Desktop für das Symbol (Lite geht auch, dann ohne Symbol); 2 GB RAM sollten reichen, 4 GB sind sicherer; ein Benutzer mit `sudo`-Recht (im Folgenden `pi`, bei dir kann er anders heißen); Internet für die Installation.

6.1 Dateien auf den Pi bringen

Der **gesamte Projektordner** `python` kommt auf den Pi, so wie er auf dem PC läuft. Auf dem Pi liegt er **in** `~/mobaledlib`:

```
/home/pi/mobaledlib/python/                 <- Projektordner (hier liegt webmobaledlib.py)
/home/pi/mobaledlib/python/pi/              <- Installationsskripte (install_pi.sh ...)
```

So entsteht es mit einer ZIP-Datei:

1. Auf dem PC den **Ordner** `python` **selbst** als ZIP packen (nicht nur seinen Inhalt), ohne `__pycache__`.
2. Die ZIP-Datei auf den Pi in den Ordner `~/mobaledlib` kopieren (`mkdir -p ~/mobaledlib`, falls es ihn nicht gibt).
3. Dort **„Hier entpacken“**: Es entsteht automatisch der Ordner `python` mit allen Dateien darunter. (Enthält die ZIP nur den Inhalt ohne Ordner, würden die Dateien lose in `~/mobaledlib` landen: dann mit „Entpacken nach …“ in einen neuen Ordner `python` entpacken.)

Alternativ im Terminal:

```
# vom Windows-PC (PowerShell); zuvor auf dem Pi:  mkdir -p ~/mobaledlib
scp -r D:\webMobaLedLib\python pi@raspberrypi.local:/home/pi/mobaledlib/
# oder auf dem Pi:  git clone <dein Repository> ~/mobaledlib       (das Repository enthält den Ordner python)
```

6.2 Installation (ein Befehl)

```
cd /home/pi/mobaledlib/python/pi
sudo bash install_pi.sh --with-arduino-ide
```

Das Skript erledigt der Reihe nach:

1. Systempakete (Python, venv, `xvfb`, ein paar Bibliotheken für die Java-IDE),
2. eine eigene Python-Umgebung `venv` mit den Paketen aus `requirements-pi.txt`,
3. Zugriff auf die USB-Anschlüsse (Gruppe `dialout`),
4. `/etc/mobaledlib-web.env` mit Port und Passwort (Voreinstellung: Benutzer `mobaledlib`, Passwort `mobaledlib`, im Programm änderbar, siehe Kapitel 8),
5. den Starter für das Desktop-Symbol und eine sudo-Regel, mit der du **nur** den Dienst `mobaledlib-web` ohne Passwort starten, anhalten und neu starten sowie den Pi herunterfahren und neu starten darfst. Dazu kommt ein kleines Hilfsprogramm (`/usr/local/sbin/mobaledlib-set-hostname`, Besitzer root), mit dem die Oberfläche den Rechnernamen ändern darf (6.7). Das Hilfsprogramm prüft den Namen streng,
6. das **Desktop-Symbol** (6.3),
7. die ARDUINO IDE 1.8.19 (nur mit `--with-arduino-ide`, etwa 150 MB Download) und die Bibliothek MobaLedLib mit ihren Abhängigkeiten,
8. den Dienst `mobaledlib-web` (startet beim Hochfahren) und einen Test der Installation (`pi_check.py`).

Option	Wirkung
`--with-arduino-ide`	IDE 1.8.19 laden, dazu `--with-arduino-libs`
`--with-arduino-libs`	nur die Bibliotheken
`--with-platformio`	PlatformIO für den Benutzer des Dienstes in `~/.platformio/penv` installieren (siehe 2.16)
`--port 9000`	anderer Port (Voreinstellung 8080)
`--hostname webMLLraspi`	Rechnernamen des Pi ändern, danach erreichbar als `http://webMLLraspi.local:8080` (6.7)
`--password GEHEIM`	anderes Anfangspasswort festlegen (Voreinstellung: `mobaledlib`)
`--local-only`	nur auf dem Pi erreichbar, kein Passwort
`--desktop-icon` / `--no-desktop-icon`	Symbol auch ohne erkannten Desktop / kein Symbol
`--no-check`	ohne den Test der Installation

Das Skript kann **mehrfach** laufen (zum Beispiel nach einem Update). Vorhandene Einstellungen bleiben erhalten. Windows-Zeilenenden (CRLF) in den Skripten entfernt das Skript selbst. Falls schon der Start mit `$'\r': command not found` abbricht: `sed -i 's/\r$//' install_pi.sh` und noch einmal starten.

6.3 Das Desktop-Symbol

Nach der Installation liegt auf dem Desktop das Symbol **„webMobaLedLib“** (und ein Eintrag im Startmenü unter Entwicklung / Elektronik). Ein Doppelklick:

1. startet den Dienst, wenn er nicht läuft (nach einem „Beenden“ im Programm zum Beispiel),
2. wartet, bis die Oberfläche bereit ist (20 bis 60 Sekunden, dabei erscheint die Meldung „webMobaLedLib wird gestartet“),
3. öffnet die Oberfläche in einem **eigenen Chromium-Fenster** ohne Adressleiste. Fehlt Chromium, nimmt er Firefox, sonst den Standardbrowser. Mit `MLL_BROWSER=<Programm>` wählst du einen anderen.

Auf dem Pi selbst brauchst du **kein Passwort** (Zugriffe von `localhost`). Aus dem Netz gilt es weiter. Fragt der Desktop beim ersten Doppelklick, ob die Datei ausgeführt werden soll, wähle „Ausführen“. Kommt nichts, starte den Starter im Terminal, dann siehst du die Meldung: `/home/pi/mobaledlib/python/pi/start_mobaledlib.sh`.

6.4 ARDUINO IDE und Bibliotheken auf dem Pi

Die allgemeinen Hinweise stehen in 2.2 und 2.3. Besonderheiten des Pi:

- Mit `--with-arduino-ide` lädt das Skript die IDE 1.8.19 für 64-Bit-ARM (`linuxaarch64`) und packt sie nach `/opt/arduino-1.8.19`. Von Hand: die Datei `arduino-1.8.19-linuxaarch64.tar.xz` laden, nach `/opt` auspacken.
- Die Bibliothek MobaLedLib installiert das Skript **ohne sudo** als der Benutzer, unter dem der Dienst läuft (sie liegt in `~/Arduino/libraries/MobaLedLib`):

  ```
  bash ~/mobaledlib/python/pi/install_arduino_libs.sh
  sudo systemctl restart mobaledlib-web
  ```

- Boards, gleich mit Version:

  ```
  bash ~/mobaledlib/python/pi/install_arduino_libs.sh --pico            # Raspberry Pi Pico (RP2040)
  bash ~/mobaledlib/python/pi/install_arduino_libs.sh --pico=5.4.2 --esp32
  ```

  (Das ESP32-Paket lässt sich auf dem Pi nicht installieren, siehe Kapitel 10; das Programm wählt es dort seit 1.90 nicht mehr aus.)

6.5 Alltag auf dem Pi

Was	Befehl
Starten	Desktop-Symbol (oder `sudo systemctl start mobaledlib-web`)
Läuft der Dienst?	`systemctl status mobaledlib-web`
Meldungen ansehen	`journalctl -u mobaledlib-web -f` (und die Datei `webmobaledlib.log` im Datenordner, 2.6)
Neu starten / anhalten	`sudo systemctl restart mobaledlib-web` / `sudo systemctl stop mobaledlib-web`
Nicht mehr beim Hochfahren starten	`sudo systemctl disable mobaledlib-web`
Installation prüfen	`cd ~/mobaledlib/python && MLL_TK_ROOT=0 venv/bin/python pi_check.py`
Herunterfahren / Neustart	Menü **Datei → Raspberry Pi herunterfahren …** (bzw. **neu starten …**), siehe 6.6
Alles wieder entfernen	`sudo bash ~/mobaledlib/python/pi/uninstall_pi.sh` (Projektordner, venv und IDE bleiben)

Der Knopf **„Beenden“** in der Oberfläche beendet das Programm. Der Dienst startet danach von selbst neu; mit dem Symbol öffnest du die Oberfläche dann wieder. Soll er aus bleiben: `sudo systemctl stop mobaledlib-web`.

6.6 Raspberry Pi herunterfahren

Ein Pi sollte nicht einfach vom Strom getrennt werden: Dabei kann die SD-Karte beschädigt werden. Die Oberfläche hat deshalb im Menü **Datei** die Einträge **Raspberry Pi herunterfahren …** und **Raspberry Pi neu starten …** (nur auf dem Pi sichtbar).

1. Eine Rückfrage fragt: **Speichern und herunterfahren**, **Ohne Speichern** oder Abbrechen.
2. Das Programm wird wie beim Knopf „Beenden“ ordentlich abgeschlossen (Tabellen und Konfiguration sichern, Arduino trennen).
3. Es erscheint die Seite „Der Raspberry Pi wird heruntergefahren“. Danach läuft `sudo systemctl poweroff`.
4. **Warte, bis die grüne LED am Pi nicht mehr blinkt** (etwa 10 Sekunden), dann kannst du den Strom trennen.

Dafür muss der Dienstbenutzer `systemctl poweroff` und `systemctl reboot` ohne Passwort ausführen dürfen. Das legt `install_pi.sh` an (sudo-Regel) und trägt `MLL_ALLOW_SHUTDOWN=1` in `/etc/mobaledlib-web.env` ein. Bei einer älteren Installation einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` ausführen. Fehlt die Regel, sagt die Oberfläche das und beendet das Programm nicht. Im Terminal geht es auch mit `sudo shutdown -h now`.

Mit einem Taster am Pi (optional): Ein Taster zwischen GPIO 3 (Pin 5) und GND (Pin 6) kann den Pi herunterfahren und wieder starten. In `/boot/firmware/config.txt` die Zeile `dtoverlay=gpio-shutdown` ergänzen und neu starten. Das Programm wird dabei nicht gefragt: Der Dienst wird von systemd beendet, nicht gespeichert. Vorher in der Oberfläche speichern.

6.7 Den Raspberry Pi im Netz finden

Von einem anderen Rechner brauchst du die Adresse des Pi. Die einfachsten Wege, in dieser Reihenfolge:

1. **Name im Netz (mDNS):** **`http://raspberrypi.local:8080`.** Der Pi meldet sich mit dem Dienst `avahi-daemon` unter `<Rechnername>.local` im Netz (auf Raspberry Pi OS vorhanden, das Installationsskript installiert ihn, falls er fehlt). Das funktioniert unter Windows 10 (neuere Versionen) und 11, macOS, iPhone und iPad und unter Linux mit `avahi` bzw. `libnss-mdns` (bei Linux Mint und Ubuntu Standard). Auf Android klappt es oft nicht. Der Rechnername ist der, den du im Raspberry Pi Imager gewählt hast (sonst `raspberrypi`). Test: `ping raspberrypi.local`.
2. **Eigener, kurzer Name** (zum Beispiel `webMLLraspi`; Groß- und Kleinschreibung spielt bei der Suche keine Rolle, erlaubt sind Buchstaben, Ziffern und `-`, höchstens 31 Zeichen). Es gibt drei Wege:
   - **Im Programm: Programm Einstellungen** → Block **„Name des Raspberry Pi im Netz“** → neuen Namen eingeben → **„Namen ändern“** und die Rückfrage bestätigen. Der Name gilt sofort, ohne Neustart. Die Oberfläche zeigt danach die neue Adresse an. Die alte Adresse gilt dann nicht mehr: Lesezeichen und das Desktop-Symbol (es benutzt `localhost` und ist nicht betroffen) entsprechend anpassen. Den Block gibt es nur auf dem Pi und nur, wenn die Installation mit `install_pi.sh` ab Version 1.06 gelaufen ist (sie legt das Hilfsprogramm und die sudo-Regel an).
   - **Bei der Installation:** `sudo bash install_pi.sh --no-check --hostname webMLLraspi`.

    - **Von Hand:** `sudo raspi-config nonint do_hostname webMLLraspi` und neu starten.
3. **Der Router:** Dort steht der Pi in der Liste der Geräte, bei der FRITZ!Box unter *Heimnetz → Netzwerk → Netzwerkverbindungen* mit seiner IP-Adresse. Die FRITZ!Box löst auch Namen auf: `http://raspberrypi.fritz.box:8080`. Empfehlung: dem Pi im Router eine **feste IP-Adresse** geben (FRITZ!Box: Gerät bearbeiten → „Diesem Netzwerkgerät immer die gleiche IPv4-Adresse zuweisen“), dann ändert sie sich nicht.
4. **Suchprogramm auf dem PC:** `install/find_mobaledlib.py` probiert zuerst `raspberrypi.local` und `mobaledlib.local` und fragt dann alle Adressen des eigenen Netzes (/24) auf dem Port 8080 ab. Die Oberfläche erkennt es an der Anmeldung „webMobaLedLib“ bzw. dem Seitentitel und zeigt Adresse und Namen. Aufruf: `python install/find_mobaledlib.py` (Windows nach der Installation: `venv_web\Scripts\python install\find_mobaledlib.py`). Andere Netze oder Ports: `--subnet 192.168.178.0/24`, `--port 9000`, `--name mein-pi`.
5. **Am Pi selbst:** `hostname -I` zeigt die IP-Adresse, `hostname` den Namen. Beide stehen auch am Ende der Installation.

7. Aktualisieren und Deinstallieren

System	Aktualisieren	Deinstallieren
alle	Menü **Hilfe → Update webMobaLedLib …** (empfohlen, siehe 7.1), oder neue Programmdateien in den Projektordner kopieren oder den Ordner `python` ersetzen (die Version steht im Dateikopf und unter dem Titel der Oberfläche), Oberfläche neu starten. **Das laufende Programm vorher beenden** (Knopf BEENDEN oben rechts): Läuft die alte Instanz noch, öffnet der Starter nur den Browser auf ihr, und es erscheint die alte Version, obwohl die Dateien neu sind. Deine Daten liegen im Datenordner und bleiben erhalten (2.6)	–
Windows	Neue Pakete: `venv_web\Scripts\pip install -U -r install\requirements-web.txt`, oder das Skript noch einmal ausführen	Verknüpfung und Ordner `venv_web` löschen
macOS	`venv_web/bin/pip install -U -r install/requirements-web.txt`, oder das Skript noch einmal ausführen	`webMobaLedLib.app` und Ordner `venv_web` löschen
Linux-Desktop	`venv_web/bin/pip install -U -r install/requirements-web.txt`, oder `install_linux.sh` noch einmal	`bash install/uninstall_linux.sh` (mit `--purge` auch `venv_web`)
Raspberry Pi	`sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` (aktualisiert auch die Python-Pakete und die Dienstdatei)	`sudo bash ~/mobaledlib/python/pi/uninstall_pi.sh`

Beim Deinstallieren bleiben Projektordner, ARDUINO IDE und Bibliotheken erhalten.

Sichern: den Datenordner (Konfiguration, Sicherungen der Tabellen, siehe 2.6) und deine Anlagendateien. Die Daten der Tabelle speicherst du mit dem Menü „Datei“ als Download im Browser (die Datei landet auf dem Rechner, an dem du sitzt). Auf dem Pi geht auch ein fertiges Abbild der SD-Karte (Raspberry Pi Imager oder `dd`).

7.1 Update über das Programm (Hilfe → Update webMobaLedLib …)

Das Programm lädt das neueste Release (`webmobaledlib.zip` aus https://github.com/haroldlinke/WebMLL/releases), vergleicht **jede Datei** mit der installierten und zeigt, was sich ändert: neue, geänderte und zu löschende Dateien, mit Versionsnummer. Erst nach **Aktualisieren** passiert etwas.

- **Nur Geändertes wird geschrieben.** Gleiche Dateien werden nicht angefasst. Das sind meist nur wenige Dateien statt des ganzen Ordners, und Virenscanner schlagen deshalb nicht an. Jede Datei wird zuerst in eine Hilfsdatei geschrieben und dann ersetzt.
- **Alles oder nichts:** Vor dem Schreiben werden die ersetzten Dateien gesichert und ein Journal angelegt

(`.update_journal.json` im Ordner `python`). Zuerst kommen alle Dateien, dann werden sie noch einmal geprüft, und **erst zuletzt** `recover.py`, `webmobaledlib.py` und die Prüfliste. Schlägt ein Schritt fehl (zum Beispiel weil ein Sicherungsprogramm eine Datei sperrt), stellt das Programm sofort den alten Stand wieder her und meldet es. Bricht das Programm mitten im Update ab (Stromausfall, Prozess beendet), macht es **beim nächsten Start selbst** das Update rückgängig, bevor irgendetwas anderes geladen wird, und zeigt oben einen Hinweis (siehe 9.6).

- **Gelöscht werden nur Dateien, die ein früheres Update installiert hat** und die im neuen Release fehlen (die Liste dazu steht in `update_manifest.json` im Datenordner). Beim **ersten Update** einer Installation gibt es diese Liste noch nicht: Dann wird nichts gelöscht. Eigene Dateien im Projektordner bleiben immer erhalten.
- **Nie angefasst** werden die Python-Umgebung (`venv`, `venv_web`), `.git`, `__pycache__`, Protokolle, Konfiguration, Passwort-Datei und die Sicherungen der Tabellen, auch wenn ein ZIP solche Dateien enthalten sollte. Eine Ausnahme sind die Dateien, die das Programm **später selbst ändert**: `LEDs_AutoProg.h` und die `.cmd`-Dateien im Ordner `LEDs_AutoProg` (`Start_Arduino.cmd`, `Start_ESP32_Sub.cmd`, `Start_PICO_Sub.cmd`, `fastBuild.cmd`, `fastBuild2.cmd`) und die Datei `platformio.ini` im Ordner `python` (sie wird beim Hochladen mit PlatformIO geschrieben, Abschnitt 2.16). Sie **gehören zum Release**: Fehlt eine, installiert das Update sie. Ist sie da, ersetzt das Update sie **nicht**, auch wenn sie vom Release abweicht, und es meldet die Abweichung nicht als Unterschied oder Fehler; ebenso die Prüfung beim Start und die Reparatur (die nur fehlende Dateien wieder herstellt). Gelöscht werden sie nie. (Andere `.cmd`-Dateien, zum Beispiel `Restart_ProgGen.cmd` und die in `hex-files`, werden wie alle Dateien behandelt.)
- **Sicherung:** Ersetzte und gelöschte Dateien liegen vorher im Datenordner unter `update_backup/<Datum-Uhrzeit>/` (die letzten 5). Zum Zurückholen die Dateien in den Projektordner zurückkopieren. Die Tabellen werden vor dem Ersetzen gesichert. Das installierte Release (ZIP) wird im Datenordner unter `release_cache` aufgehoben (die letzten zwei), damit eine Reparatur auch ohne Internet geht.
- **Python-Pakete:** Haben sich die Anforderungen geändert (`install/requirements-web.txt`, auf dem Pi `pi/requirements-pi.txt`), aktualisiert das Programm sie.
- **Name der Verknüpfungen:** Das Programm heißt seit Version 1.31 **webMobaLedLib** (vorher pyMobaLedLib). Nach einem Update über das Menü heißen Symbol und Starter noch wie bisher. Das Installationsskript (`install_windows.bat`, `install_mac.command`, `install_linux.sh`, auf dem Pi `install_pi.sh --no-check`) benennt sie um und entfernt die alten.
- **Neustart:** Nach dem Update läuft noch die alte Version. **Neu starten** sichert die Tabellen und startet das Programm neu, der offene Reiter lädt sich selbst neu. Auf dem Raspberry Pi startet der Dienst von selbst neu. Haben sich Dateien im Ordner `pi` geändert, muss dort einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` laufen (die installierte Dienstdatei ist eine Kopie).
- **Entwicklungsordner:** In einer Git-Arbeitskopie ist das Update gesperrt, damit es die Dateien nicht mit dem Release überschreibt (`MLL_ALLOW_UPDATE=1` hebt die Sperre auf). `MLL_UPDATE_URL` richtet das Update auf eine andere ZIP-Adresse.
- **Sicherheit:** Es wird nur über HTTPS geladen. Das ZIP muss den Ordner `python` mit `webmobaledlib.py` und `_web` enthalten, Pfade nach außen und Verknüpfungen werden abgelehnt, und aus dem ZIP wird nichts ausgeführt.
- **Schutz vor einem falschen oder unvollständigen ZIP:** Ein vollständiges Release enthält die **Prüfliste** `release_manifest.json` (Version, Art „vollständig“, Größe und SHA-256 jeder Datei). Das Update installiert nur, wenn die Prüfliste da ist und genau zu den Dateien im ZIP passt. Es **bricht ab, ohne etwas zu verändern**, wenn die Prüfliste fehlt (zum Beispiel bei einer Sammlung geänderter Dateien), wenn das ZIP deutlich weniger Dateien enthält als die installierte Version (weniger als 80 Prozent), wenn es mehr als 20 Dateien löschen würde oder wenn eine Datei nicht zur Prüfliste passt. Jede geschriebene Datei wird mit ihrer Prüfsumme verglichen; weicht sie ab (zum Beispiel durch ein Sicherungs- oder Virenprogramm), wird sie nicht ersetzt und als Fehler gemeldet.
- **Ältere Version:** Hat das ZIP eine **ältere** Version als die installierte, zeigt der Dialog in Rot **„ACHTUNG: Dieses ZIP hat eine ÄLTERE Version“**. Der Standardknopf ist dann **Abbrechen**; erst der Knopf **„Trotzdem auf Version … zurückgehen“** installiert sie. Das ist für einen gewollten Rücksprung gedacht; wurde nur versehentlich ein altes ZIP bereitgestellt, bitte abbrechen.
- **Meldung „konnte nicht geladen werden“:** Internetverbindung, Firewall oder Proxy prüfen, und ob das Release die Datei `webmobaledlib.zip` enthält. „Nicht ersetzt“ bei einer Datei heißt meist, dass ein anderes Programm sie gesperrt hat: Programm beenden und das Update wiederholen.

8. Zugriff aus dem Netz und Passwort

Standardmäßig ist die Oberfläche **nur auf dem eigenen Rechner** erreichbar (`http://localhost:8080`). Auf dem

Raspberry Pi ist sie im Netz erreichbar und mit einem Passwort geschützt. Beim Aufruf von einem anderen Rechner fragt der Browser nach Benutzername und Passwort.

Voreinstellung auf dem Pi: Benutzername `mobaledlib`, Passwort `mobaledlib`. Auf dem Pi selbst (Desktop-Symbol) ist kein Passwort nötig.

Passwort ändern (empfohlen, sobald der Pi im Netz erreichbar ist): In der Oberfläche **Programm Einstellungen** öffnen, im Block **„Zugriff über das Netz (Web-Oberfläche)“** das neue Passwort zweimal eingeben (mindestens 4 Zeichen) und **„Passwort ändern“** drücken. Es gilt sofort, ohne Neustart. Von einem anderen Rechner fragt der Browser beim nächsten Aufruf nach dem neuen Passwort. Solange noch das Standardpasswort gilt, zeigt der Block eine Warnung.

- Das neue Passwort steht nicht im Klartext auf dem Pi, sondern nur als Hash in der Datei `web_password.json` im Datenordner (2.6). Sie geht dem Passwort aus `/etc/mobaledlib-web.env` vor.
- **Passwort vergessen:** Entweder auf dem Pi in der Oberfläche **„Eigenes Passwort entfernen“** drücken (auf dem Pi selbst braucht man kein Passwort), oder die Datei `web_password.json` im Datenordner löschen. Dann gilt wieder das Passwort aus `/etc/mobaledlib-web.env` (Voreinstellung `mobaledlib`).
- Das Anfangspasswort lässt sich bei der Installation mit `--password` festlegen, oder später in `/etc/mobaledlib-web.env` (`MLL_PASSWORD=...`).

Auf anderen Systemen (Windows, macOS, Linux-Desktop) ist die Oberfläche standardmäßig nur auf dem eigenen Rechner erreichbar und ohne Passwort. **Zugriff aus dem Netz einschalten (ab Version 1.77):** Reiter **Programm Einstellungen**, Kasten **Zugriff aus dem Netz erlauben**, Schalter einschalten. Es muss **nichts in einer Startdatei bearbeitet** werden.

- Ist noch kein Passwort eingerichtet, fragt ein Dialog danach (mindestens 4 Zeichen; leer lassen: ohne Passwort; *Abbrechen* schaltet den Schalter zurück). Der Benutzername ist `mobaledlib`. Ohne Passwort kann jeder im Heimnetz die Oberfläche benutzen und Programme auf den angeschlossenen Arduino laden. Das Passwort lässt sich im Kasten darunter jederzeit ändern.
- Danach **Jetzt neu starten**: Das Programm sichert, startet neu, und der Browserreiter lädt sich selbst neu. Die Einstellung steht in der Datei `network.json` im Datenordner (2.6).
- **Windows:** Das Einschalten legt gleich die Firewall-Regel „webMobaLedLib 8080“ an (nur Privatnetzwerke). Windows fragt dabei nach Administratorrechten: mit *Ja* bestätigen, mehr ist nicht zu bestätigen. Lässt die Firewall den Port schon zu, fragt es nicht. Gilt das Netzwerk als „Öffentlich“, weist das Programm darauf hin, es auf „Privat“ zu stellen. Das **Ausschalten** entfernt die Regel nicht (Knopf *Freigabe entfernen* im Bereich Firewall). Die Regel steht in der Windows-Firewall unter *Eingehende Regeln* (nicht unter „Zugelassene Apps“).
- Auf dem Rechner, auf dem das Programm läuft, ist kein Passwort nötig. Von einem anderen Rechner im Browser `http://<Adresse des PCs>:8080` öffnen, Benutzername und Passwort eingeben.
- Ist die Adresse von außen vorgegeben (Startparameter `--host` oder Umgebungsvariable `MLL_HOST`), ist der Schalter gesperrt, und die Seite sagt es.

Alternativ von Hand (für Fortgeschrittene; die Vorgabe geht dem Schalter vor): vor dem Start die Umgebungsvariablen `MLL_HOST=0.0.0.0` und ein Passwort `MLL_PASSWORD=...` setzen (Benutzername `MLL_USER`, Voreinstellung `mobaledlib`). Wo:

System	Wo
Windows	`set MLL_HOST=0.0.0.0` und `set MLL_PASSWORD=...` in `install\start_windows.bat`
macOS	`export ...` in der Datei `webMobaLedLib.app/Contents/MacOS/webMobaLedLib`
Linux-Desktop	`export ...` in `install/start_linux.sh`
Raspberry Pi	in `/etc/mobaledlib-web.env` (`sudo nano /etc/mobaledlib-web.env`), danach `sudo systemctl restart mobaledlib-web`

Windows im Detail: So geht der Zugriff von einem anderen Rechner im Heimnetz. Mit dem Schalter (oben) erledigen die Schritte 1 und 2 das Programm; die Beschreibung hilft, wenn etwas nicht klappt.

1. Schalter **Zugriff aus dem Netz erlauben** einschalten und neu starten. **Von Hand statt des Schalters:** In `install\start_windows.bat` nach der Zeile `set "MLL_HEADLESS=1"` die Zeilen `set "MLL_HOST=0.0.0.0"` und `set`

`"MLL_PASSWORD=..."` einfügen. **Nicht die IP-Adresse des PCs** (zum Beispiel `192.168.178.79`) als `MLL_HOST` eintragen: Dann lauscht der Server nur auf dieser einen Adresse, und `http://localhost:8080` auf dem PC selbst geht nicht mehr. Ab Version 1.66 öffnet das Programm den Browser mit der Adresse, auf der es lauscht, in älteren Versionen kam „Seite nicht gefunden“.

2. **Windows-Firewall:** Das Einschalten des Schalters legt die Regel gleich an. Das Folgende gilt, wenn es nicht geklappt hat. Beim ersten Start fragt Windows, ob Python Verbindungen aus dem Netz zulassen darf. **Privatnetzwerke** anhaken und *Zugriff zulassen*. Wurde das weggeklickt oder gilt das Netzwerk als **Öffentlich**, antwortet der Server anderen Rechnern nicht, und deren Browser meldet einen Ladefehler. Abhilfe:

   - **Einfachster Weg (ab Version 1.67):** Reiter **Programm Einstellungen**, Bereich **Firewall**, Knopf **Zugriff aus dem Netz freigeben**. Windows fragt nach Administratorrechten (*Ja*), nach einigen Sekunden steht dort „Der Port 8080 ist freigegeben, Regel „webMobaLedLib 8080“ (Privatnetzwerke)“. Der Bereich prüft nach **Wirkung**: Er erkennt jede Regel, die eingehende Verbindungen auf den Port zulässt, auch eine von Hand angelegte oder die von Windows für Python angelegte. Regeln, deren Name mit `webMobaLedLib` beginnt (auch eine von Hand angelegte, zum Beispiel „webMobaLedLib 8080“), ersetzt oder entfernt das Programm; andere Regeln lässt es unverändert und zeigt sie an. Er warnt außerdem vor einer **Blockierregel** (gewinnt gegen jede Zulassung), wenn der Server nur auf diesem Rechner lauscht (`MLL_HOST`, Schritt 1) und wenn das Netzwerk als „Öffentlich“ eingestuft ist. **Freigabe entfernen** löscht die Regeln wieder. Die folgenden Schritte sind dasselbe von Hand. Das Programm liest die Firewall ohne Administratorrechte auf zwei Wegen (PowerShell-Befehle und die COM-Schnittstelle der Firewall). Steht dort „Der Zustand der Firewall konnte nicht gelesen werden (…)“, hat keiner der Wege geklappt; der Grund steht in Klammern und im Protokoll (**Hilfe → Logfile anzeigen**, Zeile „Firewall-Status“). Ob der Port freigegeben ist, zeigt dann nur ein Versuch vom anderen Rechner (`curl`, siehe unten).

   - Einstellungen → Netzwerk und Internet → (WLAN oder Ethernet) → Eigenschaften → Netzwerkprofil **Privat**.

   - Regel anlegen: Eingabeaufforderung **als Administrator** öffnen und eingeben (bei einem anderen Port die Nummer ersetzen; das Programm gibt den Befehl beim Start aus):

     ```
     netsh advfirewall firewall add rule name="webMobaLedLib 8080" dir=in action=allow protocol=TCP
     localport=8080 profile=private
     ```

     Entfernen: `netsh advfirewall firewall delete rule name="webMobaLedLib 8080"`. Die Regel steht in der Firewall-Oberfläche unter *Eingehende Regeln* mit der Spalte *Lokaler Port* 8080.

   - **Blockierregeln für Python:** Wurde die Rückfrage der Firewall beim ersten Start mit *Abbrechen* oder *Zugriff verweigern* beantwortet, legt Windows für `python.exe` eine Regel **Blockieren** an. Eine Blockierregel gewinnt gegen jede Zulassungsregel, auch gegen die Regel für den Port. Anzeigen (PowerShell **als Administrator**):

     ```
     Get-NetConnectionProfile
     Get-NetFirewallApplicationFilter | Where-Object { $_.Program -like "*python*" } | Get-NetFirewallRule |
     Format-Table DisplayName, Direction, Action, Enabled, Profile -AutoSize
     ```

     Die erste Zeile zeigt die Einstufung des Netzes (`NetworkCategory` muss `Private` sein, sonst `Set-NetConnectionProfile -NetworkCategory Private`). Steht in der zweiten Liste eine Zeile mit `Inbound`, `Block` und `Enabled True`, diese Regeln löschen (die Rückfrage erscheint beim nächsten Start wieder, dann *Privatnetzwerke* zulassen):

     ```
     Get-NetFirewallApplicationFilter | Where-Object { $_.Program -like "*python*" } | Get-NetFirewallRule |
     Where-Object { $_.Action -eq "Block" -and $_.Direction -eq "Inbound" } | Remove-NetFirewallRule
     ```

     Ohne PowerShell: Windows-Taste → *Windows Defender Firewall mit erweiterter Sicherheit* → *Eingehende Regeln*, nach *Name* sortieren, Einträge `python` mit Aktion *Blockieren* löschen.

   - **Andere Sicherheitssoftware** (Virenscanner mit eigener Firewall) hat eigene Regeln und muss den Port ebenfalls zulassen.

   - **Zum Eingrenzen** die Windows-Firewall für kurze Zeit ausschalten (`netsh advfirewall set allprofiles`

`state off` als Administrator), den Test wiederholen und **sofort wieder einschalten** (`netsh advfirewall set allprofiles state on`). Geht es ohne Firewall, ist sie die Ursache; dann die Regeln oben prüfen.

3. **Prüfen**, in dieser Reihenfolge:
   - Auf dem PC: `netstat -ano | findstr :8080` zeigt eine Zeile mit `LISTENING` (bei `MLL_HOST=0.0.0.0` mit `0.0.0.0:8080`). Fehlt sie, läuft der Server nicht oder auf einem anderen Port.
   - Auf dem PC im Browser `http://192.168.178.79:8080` (die eigene Adresse, `ipconfig`). Geht das, läuft der Server richtig.
   - Auf dem anderen Rechner die Verbindung zum Port testen:
     - Windows, **PowerShell** (nicht die Eingabeaufforderung): `Test-NetConnection 192.168.178.79 -Port 8080`. `TcpTestSucceeded : False` heißt: keine Verbindung.
     - Windows 10 und 11, jede Eingabeaufforderung: `curl -v --max-time 5 http://192.168.178.79:8080`.
     - Linux und macOS: `curl -v --max-time 5 http://192.168.178.79:8080` oder `nc -vz 192.168.178.79 8080`.

     Antwortet der Server (`HTTP/1.1 401 Unauthorized` bei gesetztem Passwort, sonst Seitentext, oder `succeeded`), ist die Verbindung in Ordnung. Bei **Zeitüberschreitung** (`timed out`, `TcpTestSucceeded : False`), obwohl es auf dem PC selbst geht, werden die Pakete verworfen: Firewall (Schritt 2) oder die Geräte sehen sich nicht (Gastnetz der FRITZ!Box, „Geräte dürfen untereinander kommunizieren“ ausgeschaltet). Bei **Connection refused** lauscht nichts auf dieser Adresse und diesem Port (anderer Port, oder `MLL_HOST` stimmt nicht). `ping` taugt nicht zum Prüfen: Windows beantwortet es standardmäßig nicht.
   - Dann im Browser des anderen Rechners `http://192.168.178.79:8080`, Benutzername `mobaledlib`, Passwort wie `MLL_PASSWORD`.
4. Die Adresse des PCs ändert sich, wenn der Router sie neu vergibt. Im Router dem PC eine **feste IP-Adresse** geben (FRITZ!Box: Heimnetz → Netzwerk → Gerät bearbeiten).

Die Datei auf dem Pi sieht so aus:

```
MLL_HOST=0.0.0.0        # 127.0.0.1 = nur auf dem Pi selbst
MLL_PORT=8080
MLL_USER=mobaledlib
MLL_PASSWORD=mobaledlib # Anfangspasswort; ohne diese Zeile ist die Oberfläche im Netz für jeden offen
# MLL_LOCAL_NO_PASSWORD=0   # mit 0 gilt das Passwort auch auf dem Pi selbst
```

Nach einer Änderung des Ports einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` aufrufen, damit das Symbol den neuen Port kennt.

Sicherheit: Wer die Oberfläche erreicht, kann Programme auf den angeschlossenen Arduino laden (und auf dem Pi den Rechner herunterfahren). Das Passwort wird ohne Verschlüsselung (kein https) übertragen, und das Standardpasswort ist allgemein bekannt. Nutze den Zugriff nur im eigenen Heimnetz, ändere das Passwort und mach den Rechner nicht aus dem Internet erreichbar. Sicherer ist `MLL_HOST=127.0.0.1` und der Zugriff über einen SSH-Tunnel: `ssh -L 8080:localhost:8080 pi@raspberrypi.local`, danach im Browser `http://localhost:8080`.

9. Fehlersuche

9.1 Alle Systeme

Beobachtung	Ursache und Abhilfe
„Port 8080 ist belegt“	Läuft die Oberfläche schon? Dann öffnet das Symbol nur den Browser. Sonst anderer Port (`--port 8081` beim Start, auf dem Pi in der env-Datei)
Browser öffnet sich nicht	Das Programm öffnet den Standardbrowser und probiert weitere Browser (2.4). Im Protokoll steht, welcher Weg geklappt hat (`Browser geöffnet (…)`) oder dass keiner ging. Einen bestimmten Browser erzwingt `MLL_BROWSER=brave-browser` (oder `google-chrome`); Standardbrowser unter Linux prüfen mit `xdg-settings get default-web-browser`. Sonst Adresse von Hand eingeben: `http://localhost:8080`. Der Start dauert auf langsamen Rechnern bis zu einer Minute, bevor der Browser sich öffnet: nicht noch einmal auf das Symbol klicken. Der Starter erkennt, dass die erste Instanz noch startet (Sperre im Datenordner), wartet bis zu 4 Minuten auf sie und öffnet dann den Browser
Nach dem Aktualisieren zeigt das Programm noch die alte Version (Titel der Oberfläche), auch nach einer Neuinstallation	Die alte Instanz läuft noch auf Port 8080, und der Starter öffnet nur den Browser auf ihr. Ab Version 1.25 sagt der Starter das (Terminal, Meldung auf dem Bildschirm und Protokoll: „ACHTUNG: Auf Port 8080 läuft noch eine Instanz …“). Beenden: Knopf BEENDEN, sonst Linux/macOS `ps aux | grep webmobaledlib` und `kill <Nummer>` (oder `pkill -f`

	`webmobaledlib.py`), Windows `netstat -ano | findstr :8080` und `taskkill /PID <Nummer> /F`. Danach neu starten. Welche Version gerade läuft, zeigt `http://localhost:8080/mll-version`
Meldung „Connection lost“, die Seite baut sich immer wieder neu auf	Meist ein **langsamer Rechner**: Der Aufbau der Seite dauert Sekunden, und der Browser braucht danach noch einmal 10 bis 30 s, bis er sich mit dem Server verbindet. Der Server wartet nur die Zeit `MLL_RECONNECT_TIMEOUT` (Standard 60 s) auf den Browser; ist sie zu kurz, wird die Verbindung verworfen und die Seite neu geladen. Abhilfe: einen größeren Wert setzen (zum Beispiel `MLL_RECONNECT_TIMEOUT=180 install/start_linux.sh`), einen leichteren Browser benutzen, andere Programme schließen. Die Seiten Einstellungen und Farb Test werden erst beim ersten Öffnen aufgebaut (`MLL_LAZY_TABS=0` baut alles sofort auf), das verkürzt den Start auf langsamen Rechnern deutlich. Zur Diagnose im Protokoll (2.7) die Zeilen `Seitenaufbau: …` (wo die Zeit vergeht), `Browser verbunden` und `Browser getrennt` ansehen. Läuft der Server nicht mehr, im Terminal starten (Linux/Mac `install/start_linux.sh`, Windows `install\start_windows.bat`); `webmobaledlib.log` und `webmobaledlib-crash.log` (Datenordner, 2.6) schicken. Zum Vergleich mit echtem Tk (Linux, Mac, Windows mit Bildschirm): `MLL_TK_ROOT=1 install/start_linux.sh`
Nach jeder Änderung bleibt die Tabelle einen Moment **weiß**, bevor sie wieder erscheint (langsamer Rechner)	Seit Version 1.49 bekommt der Browser bei einer Änderung nur noch die geänderten Zeilen, bei älteren Versionen die ganze Tabelle: **aktualisieren** (Hilfe → Update). Außerdem im Browser die **Hardwarebeschleunigung** prüfen (Linux: 9.4), Fenster und Browser-Zoom verkleinern, das kompakte Layout wählen (`MLL_LAYOUT=compact`). Mit `MLL_DEBUG_TIMING=1` steht im Protokoll, wie lange Aufbau und Senden dauern (`TIMING:`), das Protokoll bitte mitschicken
„Die Arduino Entwicklungsumgebung ist nicht … installiert“	IDE **1.8.19** installieren (2.2), ggf. den Pfad in „ARDUINO Einstellungen“ eintragen
Kompilieren: „MobaLedLib.h: No such file“ oder Bibliothek/Board fehlt	Bibliotheken installieren (2.3), in der Tabelle „Libraries“ prüfen
„Fehler beim Lesen der Datei …/preferences.txt“ (zum Beispiel bei „Check actual versions“)	Die Einstellungsdatei der IDE fehlt auf einem frischen System (Linux, macOS, Pi). Das Programm legt sie beim Start an. Von Hand (Linux/Pi): `mkdir -p ~/.arduino15 ~/Arduino/libraries; printf 'sketchbook.path=%s/Arduino\ntarget_package=arduino\ntarget_platform=avr\nboard=uno\n' "$HOME" > ~/.arduino15/preferences.txt`
Board installieren: `NullPointerException at … BaseNoGui.onBoardOrPortChange`	Die IDE hat ein nicht installiertes Board ausgewählt. Das Programm und die Skripte setzen die Auswahl auf `arduino:avr:uno` zurück. Von Hand (Linux/Pi): `sed -i -E '/^(target_package|target_platform|board|custom_[A-Za-z0-9_.]+)=/d' ~/.arduino15/preferences.txt; printf 'target_package=arduino\ntarget_platform=avr\nboard=uno\n' >> ~/.arduino15/preferences.txt`, dann wiederholen
Board `esp32:esp32` installieren: `Tool esptool_py is not available for your operating system`	Für diese Version des ESP32-Pakets fehlen Werkzeuge für die Prozessorart (64-Bit-ARM), siehe Kapitel 10
Pakete lassen sich nicht installieren	Internetverbindung, Firmenproxy; im Fenster steht die Fehlermeldung von `pip`
Beim Start: „Fehler beim Aktualisieren aufgetreten“, Bibliotheken oder Boards fehlen	Die Frage kommt im Browser (2.3). Die Wiederholung lädt nur, was fehlt, und endet ohne Fortschritt mit einer Meldung (ab 1.88). Die Ausgabe der IDE steht im Protokollfenster des Befehls und im Protokoll (Hilfe → Logfile anzeigen). Platinen sind groß: Internet und Platz auf dem Datenträger prüfen
ESP32 prüfen oder hochladen: `exec: "python": executable file not found in $PATH`	Die ESP32-Platine 1.0.4 braucht den Befehl `python`, neuere macOS-Versionen und Debian haben nur `python3`. Seit Version 1.89 legt das Programm ihn beim Start selbst an (`bin/python` im Datenordner, 2.3). Das Programm neu starten; fehlt die Datei dort, steht der Grund im Protokoll
ESP32 hochladen bleibt bei `Connecting........_____` stehen und endet mit `A fatal error occurred: Failed to connect to ESP32: Timed out waiting for packet header`	Der Chip kommt nicht in den Bootloader: auf dem ESP32-Board die Taste **BOOT** (IO0) **gedrückt halten**, sobald im Fenster „Connecting…“ erscheint, bis „Writing at …“ kommt. Viele DevKit-Nachbauten haben keinen zuverlässigen Auto-Reset (ein Kondensator von 10 µF zwischen EN und GND behebt das dauerhaft). Hilft es nicht: ein Datenkabel (kein reines Ladekabel) direkt am Rechner ohne Hub, einen anderen USB-Port, bei zwei Anschlüssen desselben Boards (`/dev/cu.usbserial-…` und `/dev/cu.SLAB_USBtoUART`) den anderen wählen. Das Programm weist vor dem Hochladen darauf hin
Beim Installieren der Python-Pakete: `Failed to build <Paket>` oder `Failed to build`	Für dieses System gibt es keine fertige Datei (Wheel) des Pakets, und pip versucht, es zu übersetzen, was meist scheitert. Häufigste Ursachen: 32-Bit-System, sehr altes oder sehr neues Python, kein Internet. Das Linux-Skript zeigt eine Diagnose (Prozessor, Python, pip,

`installable wheels`	glibc): bitte mitschicken. Pillow wird nicht mehr gebraucht (`requirements-web.txt` ab Version 1.01); bei einer älteren Fassung die Zeile `Pillow` aus `install/requirements-web.txt` löschen und das Skript noch einmal ausführen
„ARDUINO IDE nicht gefunden“ (Protokoll, Meldung beim Start, kein Kompilieren möglich)	Siehe 2.2. Die IDE 1.8.19 nach `~/arduino-1.8.19` legen, oder in „ARDUINO Einstellungen“ **IDE suchen** benutzen bzw. den vollständigen Pfad der Startdatei `arduino` eintragen. Es muss die IDE **1.8.x** sein, nicht 2.x. Die Orte, an denen gesucht wurde, stehen im Protokoll
`ModuleNotFoundError: No module named 'xyz'`	`venv_web/bin/pip install xyz` (Windows `venv_web\Scripts\pip`, Pi `venv/bin/pip`)
Konfiguration wird nicht gespeichert	Ist der Datenordner (2.6) beschreibbar? Ist er es nicht, nimmt das Programm einen Ersatzordner im temporären Verzeichnis und warnt im Protokoll. Anderen Ort einstellen: `MLL_DATA_DIR`

9.2 Windows

Beobachtung	Ursache und Abhilfe
„Python wurde nicht gefunden“ oder „Python wurde installiert, ist aber in diesem Fenster noch nicht bekannt“	**PC neu starten** (oder ab- und wieder anmelden) und das Skript noch einmal starten. Python von Hand: python.org, dabei „Add python.exe to PATH“ ankreuzen
„Der Computer wurde durch Windows geschützt“	*Weitere Informationen → Trotzdem ausführen*
Beim Start „Seite nicht gefunden“ unter `localhost:8080`, obwohl `MLL_HOST` gesetzt ist	`MLL_HOST` enthält eine IP-Adresse. Dann lauscht der Server nur dort. Besser den Schalter **Zugriff aus dem Netz erlauben** in den Programm Einstellungen benutzen und `MLL_HOST` nicht setzen, oder `MLL_HOST=0.0.0.0` verwenden, siehe Kapitel 8
Rote Meldung oben auf der Seite „Nicht alle Dateien passen zur Version …“	Es wurden nicht alle Dateien des Stands eingespielt. Jede Zeile nennt Datei, gefundene und verlangte Version. Die genannten Dateien in den **richtigen Ordner** kopieren (der Pfad steht davor, zum Beispiel `ExcelAPI/headless_sheet.py`); die Version steht in der ersten Zeile jeder Datei. Dieselbe Meldung steht beim Start im Konsolenfenster und im Protokoll
Ein anderer Rechner im Netz erreicht die Oberfläche nicht (Ladefehler, keine Antwort)	Meist die Windows-Firewall oder das Netzwerkprofil „Öffentlich“: Programm Einstellungen → Firewall → **Zugriff aus dem Netz freigeben** (ab 1.67), sonst Kapitel 8, Abschnitt „Windows im Detail“ (Regel von Hand, mit `netstat` und `curl` prüfen)
Fenster geht sofort wieder zu	`install\start_windows.bat` in einer Eingabeaufforderung starten und die Meldung lesen; außerdem `webmobaledlib.log` im Datenordner (2.6)
Verknüpfung fehlt auf dem Desktop	Pfad des Projektordners ohne `'` und `&`? Sonst `install\start_windows.bat` per Rechtsklick → *Senden an* → *Desktop*
IDE-Installation (`/ide`) bricht ab	Download von arduino.cc von Hand (2.2), danach `install_windows.bat /libs`

9.3 macOS

Beobachtung	Ursache und Abhilfe
„… kann nicht geöffnet werden, da es von einem nicht verifizierten Entwickler stammt“	Rechtsklick auf `webMobaLedLib` → *Öffnen* → *Öffnen* (nur beim ersten Mal)
`install_mac.command` lässt sich nicht öffnen	`chmod +x install/install_mac.command`, oder im Terminal `bash install/install_mac.command`
IDE startet nicht (Apple-Chip)	Rosetta installieren: `softwareupdate --install-rosetta`
Programm startet nichts	`install/start_mac.command` im Terminal ausführen und die Meldung lesen; `webmobaledlib-console.log` im Datenordner (2.6)

9.4 Linux-Desktop

Beobachtung	Ursache und Abhilfe
„Python 3.10 oder neuer wurde nicht gefunden“	System zu alt (Mint 20, Debian 11): neueres Python über pyenv oder deadsnakes installieren, das Skript noch einmal starten
`venv fehlgeschlagen` / `ensurepip is not available`	`sudo apt install python3-venv python3-pip`
Symbol startet nichts	`install/start_linux.sh` im Terminal ausführen und die Meldung lesen; Symbol ausführbar? (`ls -l ~/Desktop/webMobaLedLib.desktop`)
Kein serieller Anschluss / „permission denied“ auf `/dev/ttyACM0`	Gruppe `dialout`: `groups` muss `dialout` enthalten, sonst ab- und wieder anmelden (oder `sudo usermod -aG dialout $USER`)
Tabelle zeichnet sich langsam (weiße Fläche, Ruckeln)	Meist zeichnet der Browser ohne Grafikbeschleunigung auf der CPU. Chromium/Chrome: `chrome://gpu` muss bei „Canvas“ und „Rasterization“ „Hardware accelerated“ zeigen; Firefox: `about:support`, Abschnitt „Grafik“. In den Einstellungen die Hardwarebeschleunigung einschalten, bei Bedarf den Grafiktreiber installieren (`sudo ubuntu-drivers autoinstall`). Auf sehr alten Rechnern hilft ein kleineres Fenster

9.5 Raspberry Pi

Beobachtung	Ursache und Abhilfe
Symbol: „Der Dienst konnte nicht gestartet werden“	sudo-Regel fehlt: `sudo bash install_pi.sh --no-check` noch einmal, oder `sudo systemctl start mobaledlib-web`
Symbol: keine Reaktion	im Terminal `~/mobaledlib/python/pi/start_mobaledlib.sh` ausführen und die Meldung lesen; Symbol ist ausführbar? (`ls -l ~/Desktop`)
Seite lädt nicht	`systemctl status mobaledlib-web`, Adresse und Port prüfen (Port `:8080` an die Adresse anhängen), beim ersten Start 1 Minute warten
`raspberrypi.local` wird nicht gefunden	mDNS geht nicht überall: Gastnetz oder WLAN mit Geräteisolation, Firmennetz, Android. Die IP-Adresse nehmen (Router, `find_mobaledlib.py`, 6.7); auf dem Pi `systemctl status avahi-daemon`
Anmeldefenster, Passwort wird nicht akzeptiert	Benutzer `mobaledlib`; Passwort: das Standardpasswort `mobaledlib` oder das im Programm gesetzte. Vergessen: Datei `web_password.json` im Datenordner löschen (Kapitel 8)
Kein Port in der Auswahl / „permission denied“ auf `/dev/ttyACM0`	Gruppe `dialout` (der Dienst hat sie), bei Tests von Hand einmal ab- und anmelden
Kompilieren: „no X11 DISPLAY“ oder Java-Fehler	Läuft der Dienst über `xvfb-run`? (`/etc/systemd/system/mobaledlib-web.service`, `sudo apt install xvfb`)
Der Dienst startet nicht	`journalctl -u mobaledlib-web -e`; `pi_check.py` zeigt fehlende Pakete
Anmeldung am Pi per Netz geht nicht	`MLL_HOST=0.0.0.0` und Passwort in `/etc/mobaledlib-web.env` (Kapitel 8)

Der Raspberry Pi hängt sich auf, oder das Programm verschwindet ohne Meldung. Das Programm selbst bricht in solchen Fällen nicht ab: Im Protokoll (`~/.local/share/webMobaLedLib/webmobaledlib.log`) endet dann eine Zeile mitten im Satz, es folgen **Nullbytes** (in Editoren als Punkte oder Kästchen), und die Uhrzeit des nächsten Starts ist **früher** als die letzte Zeile (der Raspberry Pi hat keine Uhr mit Batterie und startet mit einer alten Zeit). Das Betriebssystem hat den Pi dann hart angehalten, meist wegen zu wenig Strom. Typische Ursachen: - **Netzteil zu schwach:** beim Pi 400 das offizielle Netzteil (5 V, 3 A) benutzen. Eine **USB-SSD oder -Festplatte** und ein **Arduino mit vielen LEDs** am USB-Anschluss belasten die Stromversorgung zusätzlich; solche Geräte besser getrennt versorgen (Arduino und LEDs mit eigenem Netzteil, SSD an einem Hub mit eigener Stromversorgung). - **Hitze:** der Pi 400 hat keinen Lüfter; bei längerer Last (Übersetzen mit der ARDUINO IDE) sollte das Gehäuse frei stehen. - **Speicher:** die ARDUINO IDE (Java) und das Programm brauchen zusammen gut 1,5 GB; weitere Programme (Browser mit vielen Tabs) schließen.

Das Programm beobachtet das ab 2.22 selbst: Im Protokoll steht beim Start eine Zeile „Raspberry Pi: Modell, get_throttled=…, Temperatur, Speicher“, danach etwa einmal pro Minute eine weitere. Meldet der Pi

Unterspannung oder **Drosselung**, steht eine Warnung im Protokoll und beim nächsten Öffnen der Seite eine gelbe Meldung. Von Hand zeigt `vcgencmd get_throttled` die Bits (0x0 ist in Ordnung; 0x50005 heißt Unterspannung, aktuell und seit dem Start), und `dmesg | grep -i -E "volt|oom|killed"` zeigt Meldungen des Systems. Bei einem Absturz bitte das Protokoll aufheben und mitschicken.

Dateien mit Umlauten fehlen nach dem Entpacken (zum Beispiel `Pattern_Config_Examples/vorsignal_grün.png`, beim Start „9 Dateien fehlen“): Ältere ZIP-Dateien (bis 2.18) enthielten die Namen mit Umlauten ohne UTF-8-Kennzeichen; Linux („Hier entpacken“) legte sie dann mit falschem Namen an. Abhilfe: **Hilfe → Update webMobaLedLib …** (stellt die Dateien wieder her). Ab 2.22 baut `zip_erstellen.bat` das ZIP mit Python und setzt das Kennzeichen; die Prüfung des ZIP lehnt ein ZIP ohne es ab.

9.6 Wenn nichts mehr geht (Installation beschädigt oder Update unterbrochen)

Ein Update kann abbrechen, wenn der Strom ausfällt, das Programm beendet wird oder ein Sicherungs- oder Virenprogramm Dateien sperrt. Das Update ist deshalb „alles oder nichts“ (siehe 7.1), und es gibt drei Hilfen:

1. **Hinweis „Das Update … ist abgebrochen oder fehlgeschlagen. Das Programm ist automatisch zur vorherigen Version … zurückgegangen“** (orange, oben auf der Seite): Das Programm hat selbst den alten Stand wiederhergestellt, Ihre Tabellen und Einstellungen sind nicht betroffen. Mit **OK** schließen. Das Update später noch einmal versuchen. Scheitert es wieder an derselben Datei, **Programm beenden, Sicherungs- oder Virenprogramm kurz anhalten** (oder den Ordner `python` und den Datenordner ausnehmen) und das Update wiederholen.

2. **Rote Meldung „… Dateien fehlen oder haben eine andere Größe als in der Prüfliste des Releases“:** Menü **Hilfe → Update webMobaLedLib …** ausführen. Auch bei gleicher Version vergleicht das Update jede Datei mit dem Release und **stellt fehlende und beschädigte Dateien wieder her**. Danach das Programm neu starten. (Dateien, die das Programm selbst erzeugt, zum Beispiel `LEDs_AutoProg.h`, werden beim Start nur auf Vorhandensein geprüft.)

3. **Das Programm startet gar nicht, oder rot „Das automatische Zurückgehen war nicht vollständig“: Reparaturskript.** Es braucht das Programm nicht und läuft auch, wenn `webmobaledlib.py` beschädigt ist. Das Programm vorher beenden, dann:

System	Aufruf
Windows	im Ordner `python\install` die Datei **`repair_windows.bat`** doppelklicken
Linux	`bash install/repair_linux.sh` im Ordner `python`
Raspberry Pi	`bash install/repair_linux.sh`, danach `sudo systemctl restart mobaledlib-web`
macOS	im Ordner `python/install` die Datei **`repair_mac.command`** doppelklicken

   Es macht ein abgebrochenes Update rückgängig, vergleicht die Dateien mit der Prüfliste (`release_manifest.json`), sucht ein vollständiges Release (zuerst die zwei zwischengespeicherten im Datenordner unter `release_cache`, sonst der Download von GitHub), prüft es und schreibt nur die fehlenden oder abweichenden Dateien neu. Die ersetzten Dateien werden vorher im Datenordner unter `repair_backup` gesichert. Es fragt vor dem Schreiben (**j**). **Ohne Internet:** `webmobaledlib.zip` von https://github.com/haroldlinke/WebMLL/releases laden und das Skript mit `--zip <Pfad zu webmobaledlib.zip>` aufrufen (unter Windows an `repair_windows.bat` anhängen, zum Beispiel in einer Eingabeaufforderung). Weitere Optionen: `--check-only` (nur prüfen), `--offline`, `--allow-older` (auch ein älteres Release), `--all` (auch erzeugte Dateien vergleichen).

4. **Das Update bricht mit „keine Prüfliste“, „unvollständig“ oder „würde … Dateien löschen“ ab:** Das bereitgestellte ZIP ist nicht das vollständige Release. Es wurde nichts verändert. Bitte dem Entwickler melden, bis das richtige ZIP da ist.

5. **Auch das Reparaturskript fehlt oder geht nicht** (von Hand):

   1. Das Programm beenden.
   2. Das aktuelle `webmobaledlib.zip` von https://github.com/haroldlinke/WebMLL/releases laden.
   3. Den Ordner `python` **umbenennen** (zum Beispiel in `python_defekt`), das ZIP entpacken (es enthält den

Ordner `python`) und wie gewohnt starten (Windows `install\start_windows.bat`, macOS `start_mac.command`, Linux `start_linux.sh`; auf dem Raspberry Pi den Dienst neu starten: `sudo systemctl restart mobaledlib-web`).

4. Ihre Tabellen, Einstellungen und das Passwort liegen im **Datenordner** (Abschnitt 2.6) und bleiben dabei erhalten. Die Python-Umgebung (`venv`, `venv_web`) gehört nicht zum ZIP; liegt sie im alten Ordner `python`, den Ordner `venv_web` (bzw. `venv`) in den neuen Ordner verschieben.

6. **Dateien zurückholen:** Ersetzte und gelöschte Dateien liegen vor jedem Update im Datenordner unter `update_backup/<Datum-Uhrzeit>/` (die letzten 5), die von der Reparatur ersetzten unter `repair_backup/<Datum-Uhrzeit>/`. Einzelne Dateien lassen sich von dort in den Ordner `python` zurückkopieren.

10. Was nicht oder noch nicht geht

- **ESP32 auf 64-Bit-ARM (Raspberry Pi, vermutlich auch Macs mit Apple-Chip):** Das Programm verlangt das ESP32-Boardpaket in der Version 1.0.4. Für diese Version bietet Espressif keine Werkzeuge für diese Prozessorart an, die Installation über die Boardverwaltung der IDE 1.8.19 scheitert mit „Tool esptool_py is not available for your operating system“. ESP32-Programme werden deshalb auf einem Windows- oder Intel-Rechner erzeugt und hochgeladen. Pico und Arduino laufen auch auf dem Pi. `python3 pi/esp32_index_check.py <Version>` zeigt, für welche Versionen alle Werkzeuge für den Rechner vorhanden sind. Eine neuere Version müsste das Projekt festlegen und testen. Seit Version 1.90 wählt das Programm das ESP32-Paket auf Linux mit 64-Bit-ARM nicht mehr automatisch aus (sonst scheiterte die Installation bei jedem Start).
- **Noch nicht umgesetzte Formulare** (es gab sie in der Tk-Version): ServoAnim, LEDAnim, LEDColorAnim, Multiple_LEDColorAnim und der Mustereditor. Sie zeigen nichts an.
- **Weitere Reiter:** Die Ausgabe beim Kompilieren und Hochladen (der ARDUINO Monitor der Tk-Fassung) erscheint im Protokollfenster. Von den DM Servo Einstellungen fehlen das Programmieren des ATtiny (Hex-Datei über den Bus) und das Formular für den Programmieradapter (2.13).
- **Patterngenerator:** Test Pattern und Senden an ein Modul, die Wirkung der Spezial Modes auf dem Modul und der Multiplexer fehlen noch (2.10).
- **Stand der Prüfung:** Ausprobiert sind
  - **Raspberry Pi** (Raspberry Pi OS Trixie, Python 3.13): Neuinstallation mit `install_pi.sh`, Dienst, Zugriff aus dem Netz, Desktop-Symbol, IDE, Bibliotheken, Boards (ohne ESP32) und das Herunterfahren.
  - **Linux Mint** (32-Bit-Debian 12, Python 3.11, sehr langsamer Laptop) und **Ubuntu 25.10** (Python 3.13): Installation mit `install_linux.sh`, Betrieb, Update über das Menü und das kompakte Layout.
  - **Windows 10** (`install_windows.bat /ide` nach dem Neustart des PCs, Python 3.12) und **Windows 11** (Python 3.14): Installation und Betrieb.
  - **macOS:** Ein Tester setzt das Programm auf einem neueren macOS ein. Sein Fehler beim Prüfen für den ESP32 (`python` fehlt) ist seit Version 1.89 behoben, die Rückmeldung dazu steht noch aus.
  - **Nicht ausprobiert:** der Rest von **macOS** (auf macOS 10.11.6 scheitert die Installation der Python-Pakete, `orjson`; neuere Versionen sind nicht getestet) und der Download und die stille Installation der IDE unter Windows und macOS. Die Skripte für macOS liefen nur unter Linux mit einer lokalen ZIP-Datei als Ersatz.
  - **Ohne Tk** (Standard ab Version 1.33): Auf dem Pi ist das seit Langem so. Auf Windows, dem Linux-Desktop und macOS wurde bis Version 1.32 ein unsichtbares echtes Tk-Fenster angelegt. Der Ersatz ist dort ab Version 1.33 der Normalfall: **Auf Windows (Entwicklungsrechner) läuft er**, auf dem Linux-Desktop und macOS ist er noch wenig ausprobiert. Rückmeldungen sind willkommen (Protokollzeile `Tk: …`).
  - **32-Bit-Systeme** funktionieren nur, wenn für alle Python-Pakete fertige Dateien vorliegen (Pillow wird nicht mehr gebraucht). Auf dem Mint-Laptop war das der Fall.
- **Update über das Programm** gibt es ab Version 1.27. Beim ersten Update einer Installation werden keine veralteten Dateien gelöscht (7.1).
- **Kompaktes Layout:** Die Zeilenhöhe der Tabelle bleibt unverändert (22 Pixel), Dialoge sind nur über Schrift und Abstände angepasst.
- **Mehrsprachigkeit:** Die Texte, die das Web-Programm selbst ergänzt (Menüs, Dialoge, Einstellungen, Meldungen), sind nur deutsch. Die Mehrsprachigkeit folgt später.

Bei einer Fehlermeldung bitte den Text aus dem Installationsfenster und die Dateien `webmobaledlib.log` und `webmobaledlib-crash.log` aus dem Datenordner (2.6) schicken.